



**RV College of
Engineering®**

Go, change the world

Rashtreeya Sikshana Samithi Trust
RV College of Engineering®, Bengaluru.
(An Autonomous Institution Affiliated to VTU, Belagavi)

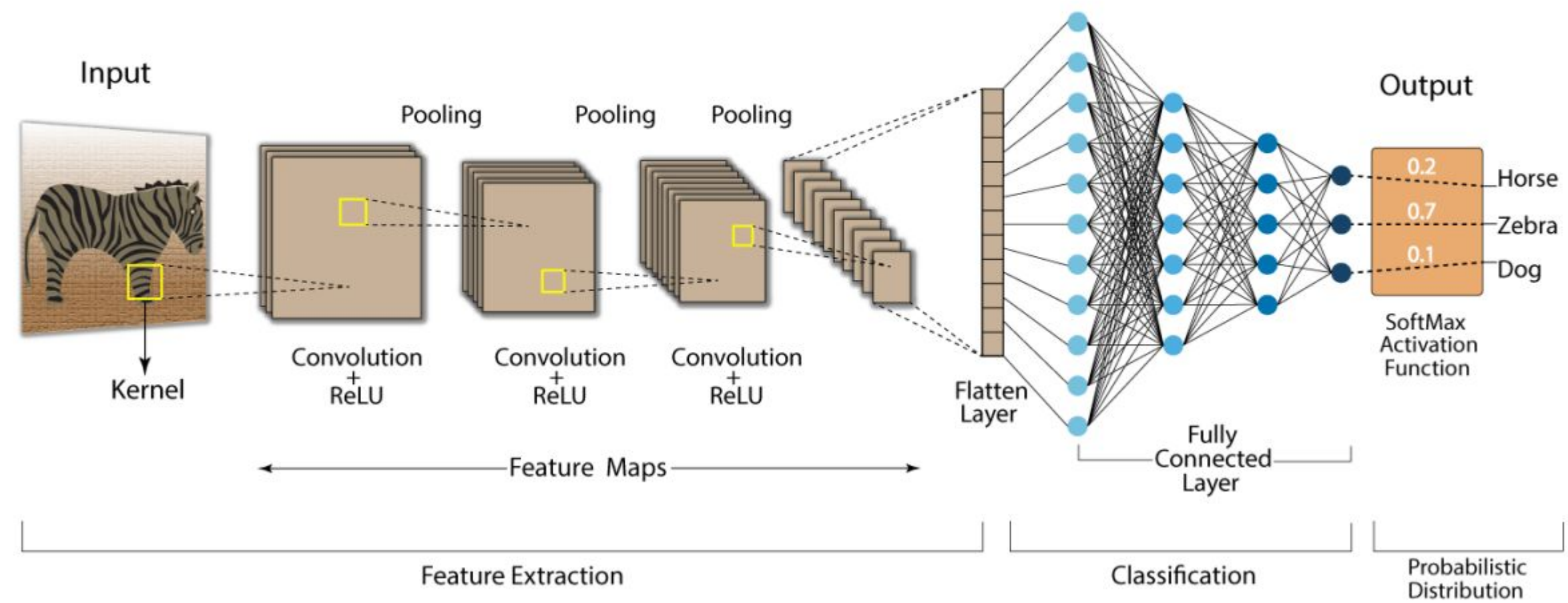
DEEP LEARNING

UNIT 2

Convolution Neural Networks

- A **convolutional neural network (CNN)** is a type of artificial neural network used for processing data with a grid-like topology, such as images.
- CNNs automatically learn to identify and classify objects and patterns by applying a series of filters in convolutional layers, which are followed by pooling and fully connected layers.
- They are widely used in computer vision tasks like image recognition and segmentation, but also in fields like natural language processing.

Convolution Neural Network (CNN)



Convolutional neural networks or CNNs, are a specialized kind of neural network for processing data that has a known grid-like topology.

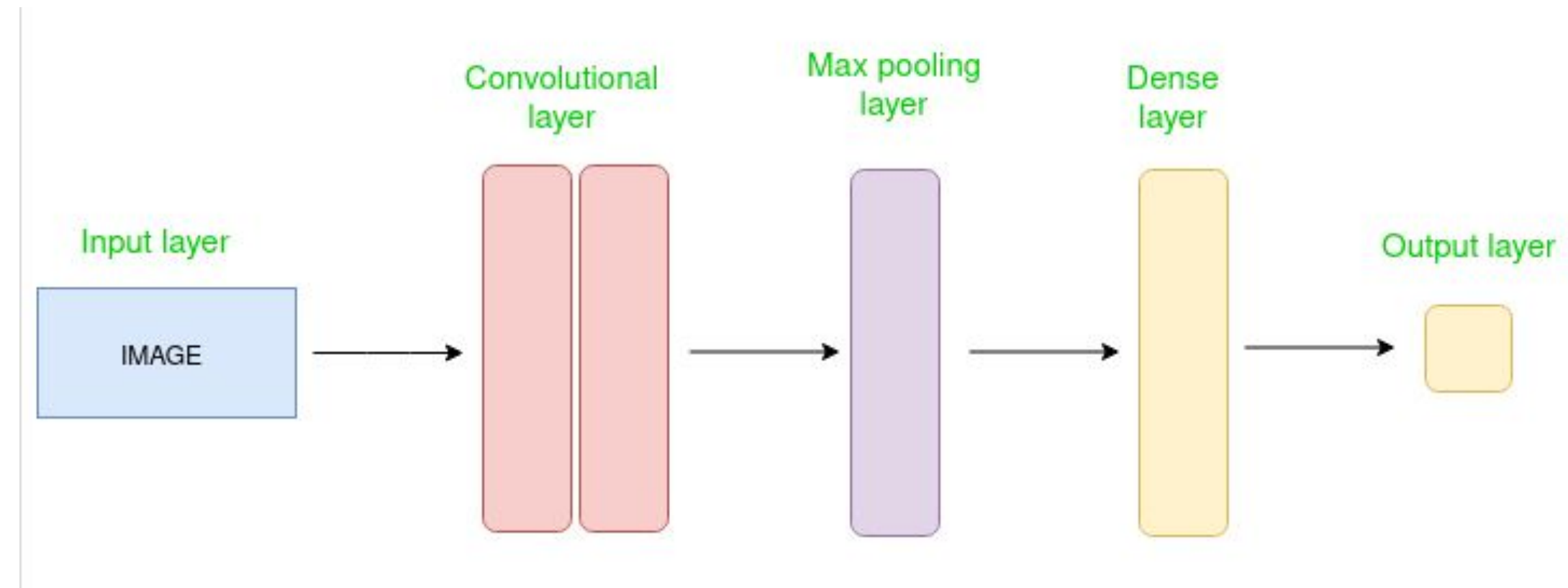
Examples **include time-series data**, which can be thought of as a **1-D grid** taking samples at regular time intervals, and image data, which can be thought of as a 2-D grid of pixels.

The name “convolutional neural network” indicates that the **network employs a mathematical linear operation called convolution**. Convolutional networks are simply neural networks that use convolution in place of general matrix multiplication in at least one of their layers.

Convolutional networks stand out as an example of neuroscientific principles influencing deep learning.

The goal of this chapter is to describe the kinds of tools that convolutional networks provide

- CNN is composed of multiple layers
 - Input layer
 - Convolutional layer
 - Pooling layer
 - Fully connected layers.



A Simple CNN Architecture

How Convolution Layer works?

- This is the first building block of a CNN.
- As the name suggests, the main mathematical task performed is called convolution, which is the application of a sliding window function to a matrix of pixels representing an image.
- The sliding function applied to the matrix is called kernel or filter, and both can be used interchangeably.
- In the convolution layer, several filters of equal size are applied, and each filter is used to recognize a specific pattern from the image, such as the curving of the digits, the edges, the whole shape of the digits, and more.
- In convolution layer, we use small grids (called filters or kernels) that move over the image. Each small grid is like a mini magnifying glass that looks for specific patterns in the photo, like lines, curves, or shapes.
- As it moves across the photo, it creates a new grid that highlights where it found these patterns.

Input (4×4):

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}$$

Kernel (3×3):

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Bias = 5

Output size formula (no padding):

$$O = \frac{(N - F)}{S} + 1$$

where

 $N = 4$ (input size), $F = 3$ (kernel size), $S = 1$ (stride).

$$O = \frac{(4 - 3)}{1} + 1 = 2$$

✓ Output will be 2×2

We'll slide the 3×3 kernel across the input.

(1,1) position:

Input patch:

$$\begin{bmatrix} 1 & 2 & 3 \\ 5 & 6 & 7 \\ 9 & 10 & 11 \end{bmatrix}$$

Multiply elementwise with kernel and sum:

$$\begin{aligned} (0 \times 1) + (-1 \times 2) + (0 \times 3) + (-1 \times 5) + (5 \times 6) + (-1 \times 7) + (0 \times 9) + (-1 \times 10) + (0 \times 11) \\ = 0 - 2 + 0 - 5 + 30 - 7 + 0 - 10 + 0 = 6 \end{aligned}$$

Add bias = 6 + 5 = 11

✅ Output(1,1) = 11

Final Output Feature Map

$$\text{Output} = \begin{bmatrix} 11 & 12 \\ 15 & 16 \end{bmatrix}$$

Mathematical Overview of Convolution

- Convolution layers consist of a set of learnable filters (or kernels) having small widths and heights and the same depth as that of input volume (3 if the input layer is image input).
- For example, if we have to run convolution on an image with dimensions $34 \times 34 \times 3$. The possible size of filters can be $a \times a \times 3$, where 'a' can be anything like 3, 5, or 7 but smaller as compared to the image dimension.
- During the forward pass, we slide each filter across the whole input volume step by step where each step is called stride (which can have a value of 2, 3, or even 4 for high-dimensional images) and compute the dot product between the kernel weights and patch from input volume.
- As we slide our filters we'll get a 2-D output for each filter and we'll stack them together as a result, we'll get output volume having a depth equal to the number of filters. The network will learn all the filters.

Input Kernel Bias Output

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

★

0	-1	0
-1	5	-1
0	-1	0

+

5

=

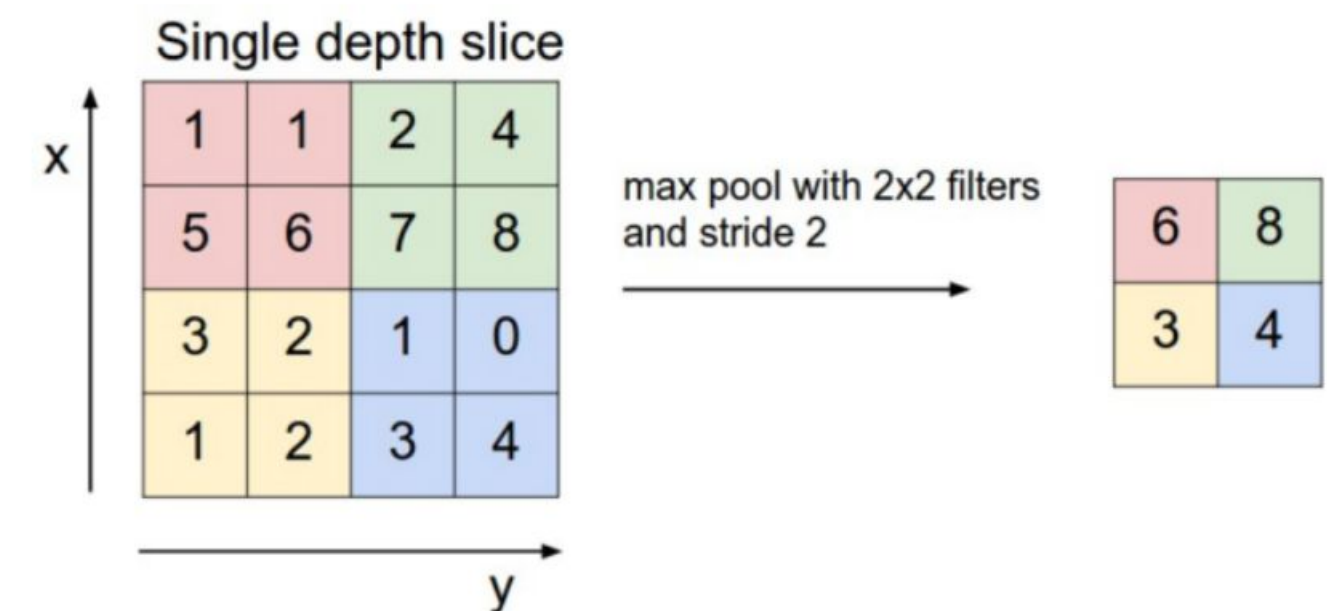
11	12
15	16

★ Cross-Correlation

- A complete Convolution Neural Networks architecture is also known as convnets. A convnets is a sequence of layers, and every layer transforms one volume to another through a differentiable function.
- Let's take an example by running a convnets on of image of dimension $32 \times 32 \times 3$.
- **Input Layers:** It's the layer in which we give input to our model. In CNN, Generally, the input will be an image or a sequence of images. This layer holds the raw input of the image with width 32, height 32, and depth 3.
- **Convolutional Layers:** This is the layer, which is used to extract the feature from the input dataset. It applies a set of learnable filters known as the kernels to the input images. The filters/kernels are smaller matrices usually 2×2 , 3×3 , or 5×5 shape. it slides over the input image data and computes the dot product between kernel weight and the corresponding input image patch. The output of this layer is referred as feature maps. Suppose we use a total of 12 filters for this layer we'll get an output volume of dimension $32 \times 32 \times 12$.

- **Activation Layer**: By adding an activation function to the output of the preceding layer, activation layers add nonlinearity to the network. it will apply an element-wise activation function to the output of the convolution layer. Some common activation functions are **RELU**: $\max(0, x)$, **Tanh**, **Leaky RELU**, etc. The volume remains unchanged hence output volume will have dimensions $32 \times 32 \times 12$.
- **Pooling layer**: This layer is periodically inserted in the convnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents overfitting. Two common types of pooling layers are **max pooling** and **average pooling**. If we use a max pool with 2×2 filters and stride 2, the resultant volume will be of dimension $16 \times 16 \times 12$.

- **Flattening:** The resulting feature maps are flattened into a one-dimensional vector after the convolution and pooling layers so they can be passed into a completely linked layer for categorization or regression.
- **Fully Connected Layers:** It takes the input from the previous layer and computes the final classification or regression task.
- **Output Layer:** The output from the fully connected layers is then fed into a logistic function for classification tasks like sigmoid or softmax which converts the output of each class into the probability score of each class.



When you define the network, the convolved features are controlled by three parameters:

1. **Depth:** It defines the number of filters to apply during the convolution. In the previous example, you saw a depth of 1, meaning only one filter is used. In most of the case, there is more than one filter.
2. **Stride:** It defines the number of "pixel's jump" between two slices. If the stride is equal to 1, the windows will move with a pixel's spread of one. If the stride is equal to two, the windows will jump by 2 pixels. If you increase the stride, you will have smaller feature maps
3. **Zero-padding:** A padding is an operation of adding a corresponding number of rows and column on each side of the input features maps. In this case, the output has the same dimension as the input.

The Convolution Operation : In general form, convolution is an operation on two functions of a real-valued argument.

Suppose we are tracking the location of a spaceship with a laser sensor. Our laser sensor provides a single output $x(t)$, the position of the spaceship at time t . Both x and t are real valued, that is, we can get a different reading from the laser sensor at any instant in time.

Now suppose that our laser sensor is somewhat noisy. To obtain a less noisy estimate of the spaceship's position, we would like to average several measurements. Of course, more recent measurements are more relevant, so we will want this to be a weighted average that gives more weight to recent measurements.

We can do this with a weighting function $w(a)$, where a is the age of a measurement. If we apply such a weighted average operation at every moment, we obtain a new function s providing a smoothed estimate of the position of the spaceship:

$$s(t) = \int x(a)w(t-a)da.$$

$$s(t) = (x * w)(t).$$

This operation is called convolution. The convolution operation is typically denoted with an Asterisk:

In convolutional network terminology, the first argument (in this example, the function x) to the convolution is often referred to as the input , and the second argument (in this example, the function w) as **the kernel** . The output is sometimes referred to as **the feature map**.

when we work with data on a computer, time will be discretized, The time index t can then take on only integer values. If we now assume that x and w are defined only on integer t , we can define the discrete convolution:

$$s(t) = (x * w)(t) = \sum_{a=-\infty}^{\infty} x(a)w(t-a).$$

if we use a two-dimensional image I as our input, we probably also want to use a two-dimensional kernel K

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n)K(i-m, j-n).$$

Convolution is commutative, meaning we can equivalently write

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i-m, j-n)K(m, n).$$

The commutative property of convolution arises because we **have flipped** the kernel relative to the input, in the sense that as m increases, the index into the input increases, but the index into the kernel decreases.

The only reason to flip the kernel is to obtain the commutative property. While the commutative property is useful for writing proofs, it is not usually an important property of a neural network implementation. Instead, many neural network libraries implement a related function called **the cross-correlation**, which is the same as convolution but without flipping the kernel.

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n).$$

Discrete convolution can be viewed as multiplication by a matrix, but the matrix has several entries constrained to be equal to other entries. For example, for univariate discrete convolution, each row of the matrix is constrained to be equal to the row above shifted by one element. This is known as a **Toeplitz matrix**. In two dimensions, a doubly block **circulant matrix** corresponds to convolution

◆ 1. Convolution as a matrix operation

Normally, we write 1D discrete convolution like this:

$$s[n] = \sum_m x[m] w[n - m]$$

But we can also think of it as a **matrix multiplication**:

$$\mathbf{s} = \mathbf{T}_w \mathbf{x}$$

where $\mathbf{T}_w$ is a *special kind of matrix* built from the kernel w .

◆ 2. Example (1D case)

Suppose your **input** has 4 elements:

$$x = [x_1, x_2, x_3, x_4]$$

and your **kernel** (filter) has 3 elements:

$$w = [w_1, w_2, w_3]$$

Then the convolution result can be written as:

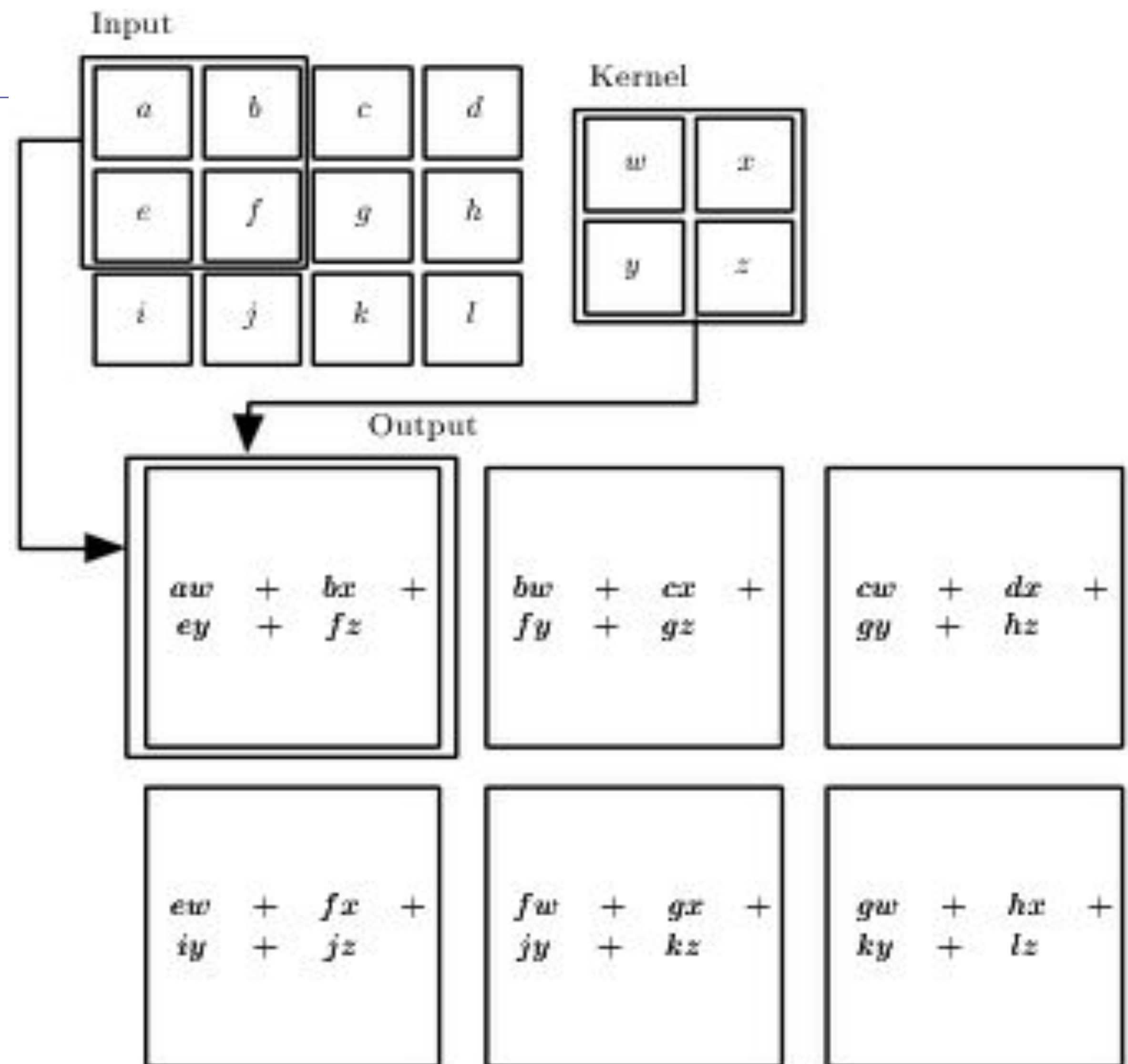
$$\begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \\ s_6 \end{bmatrix} = \begin{bmatrix} w_1 & 0 & 0 & 0 \\ w_2 & w_1 & 0 & 0 \\ w_3 & w_2 & w_1 & 0 \\ 0 & w_3 & w_2 & w_1 \\ 0 & 0 & w_3 & w_2 \\ 0 & 0 & 0 & w_3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

That big matrix is called a **Toeplitz matrix**.

- A **Toeplitz matrix** is one where **each diagonal has the same value**.
- Here, every row looks like the one above it, but *shifted one step to the right*:
- Row 1: $[w_1, 0, 0, 0]$
- Row 2: $[w_2, w_1, 0, 0]$
- Row 3: $[w_3, w_2, w_1, 0]$
- “Each row of the matrix is constrained to be equal to the row above shifted by one element.”

- In 2D, convolution can still be seen as matrix multiplication, but now the kernel is applied over both rows and columns.
- The resulting large matrix is called a **doubly block circulant matrix** — it's built from smaller Toeplitz blocks arranged in a circulant (rotating) way.

Figure 9.1: An example of 2-D convolution without kernel flipping. We restrict the output to only positions where the kernel lies entirely within the image, called “valid” convolution in some contexts. We draw boxes with arrows to indicate how the upper-left element of the output tensor is formed by applying the kernel to the corresponding upper-left region of the input tensor



Convolution leverages three important ideas that can help improve a machine learning system: sparse interactions (also referred to as sparse connectivity or sparse weights), parameter sharing and equivariant representations

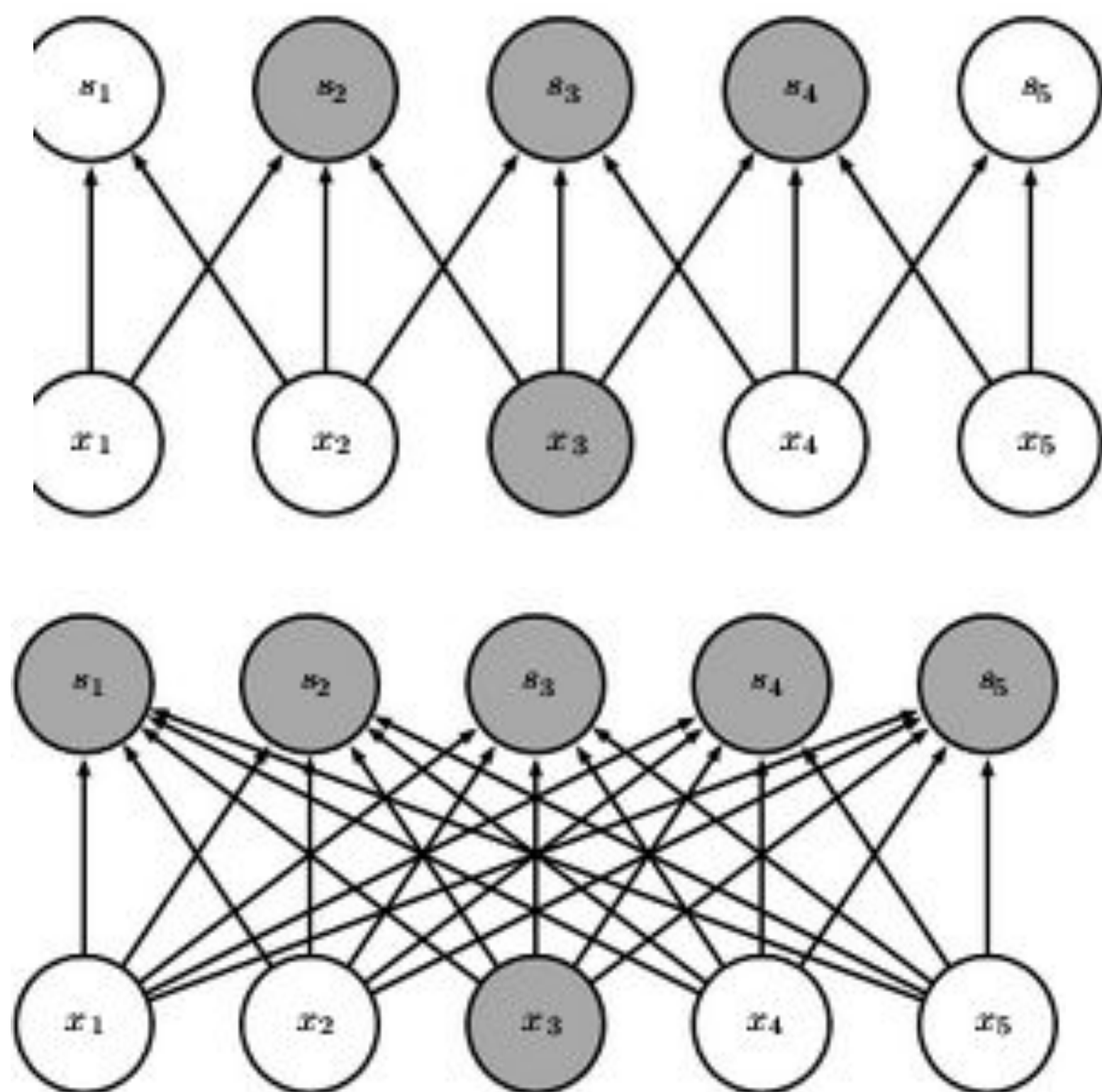


Figure 9.2: Sparse connectivity, viewed from below. We highlight one input unit, x_3 , and highlight the output units in s that are affected by this unit.

(Top) When s is formed by convolution with a kernel of width 3, only three outputs are affected by x .

(Bottom) When s is formed by matrix multiplication, connectivity is no longer sparse, so all the outputs are affected by x_3 .

- For example, when processing an image, the input image might have thousands or millions of pixels, but we can detect small, meaningful features such as edges with kernels that occupy only tens or hundreds of pixels.
- This means that we need to store fewer parameters, which both reduces the memory requirements of the model and improves its statistical efficiency. It also means that computing the output requires fewer operations.
- These improvements in efficiency are usually quite large. If there are m inputs and n outputs, then matrix multiplication requires $m \times n$ parameters and the algorithms used in practice have $O(m \times n)$ runtime (per example).
- If we limit the number of connections each output may have to k , then the sparsely connected approach requires only $k \times n$ parameters and $O(k \times n)$ runtime.

If there are m inputs and n outputs, then matrix multiplication requires $m \times n$ parameters, and the algorithms used in practice have $O(m \times n)$ runtime (per example). If we limit the number of connections each output may have to k , then the sparsely connected approach requires only $k \times n$ parameters and $O(k \times n)$ runtime.

For many practical applications, it is possible to obtain good performance on the machine learning task while keeping k several orders of magnitude smaller than m .

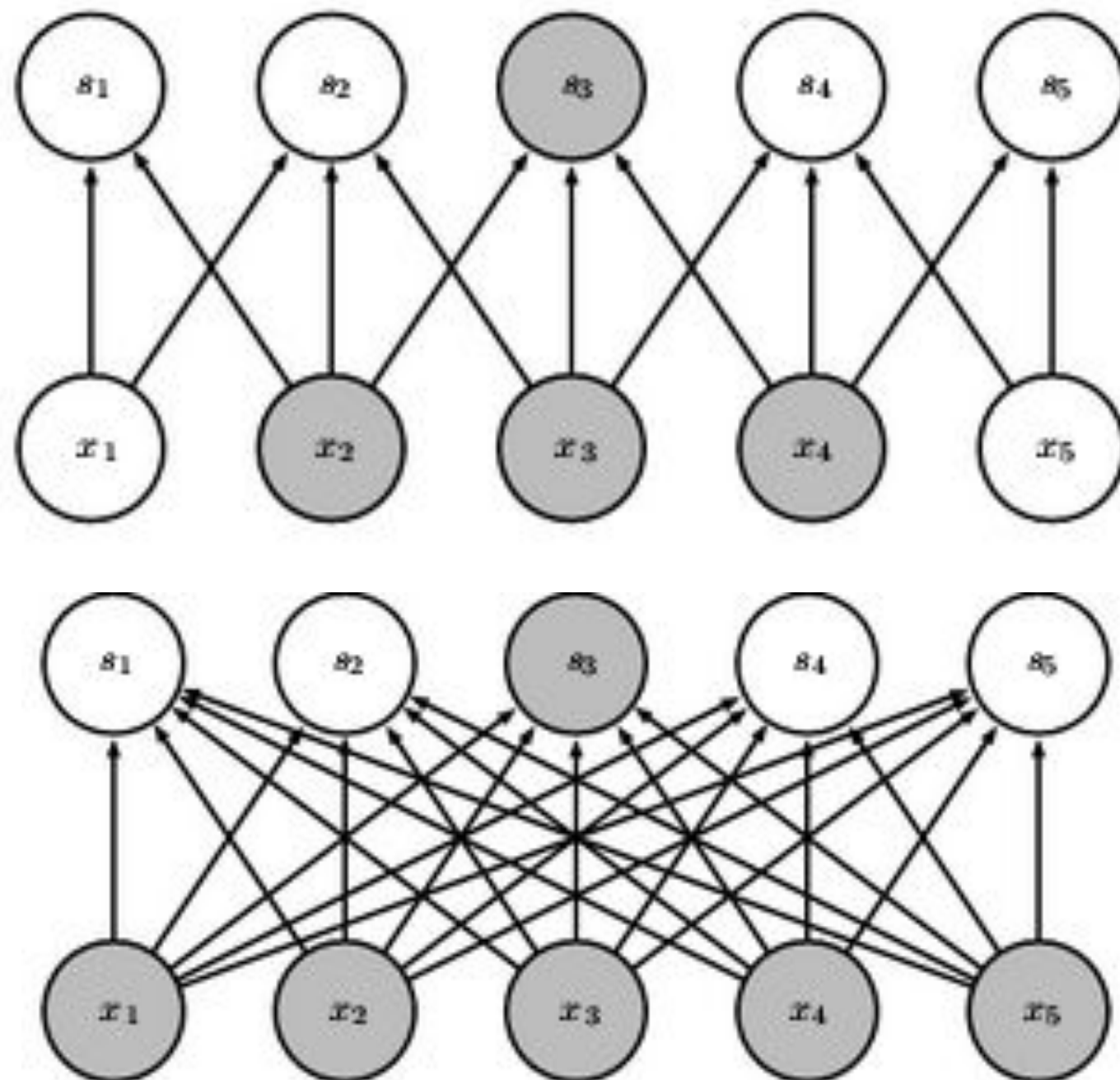


Figure 9.3: Sparse connectivity, viewed from above. We highlight one output unit, s_3 , and highlight the input units in x that affect this unit. These units are known as the receptive field of s_3 .

(Top) When s is formed by convolution with a kernel of width 3, only three inputs affect s_3 .

(Bottom) When s is formed by matrix multiplication, connectivity is no longer sparse, so all the inputs affect s_3 .

In a deep convolutional network, units in the deeper layers may indirectly interact with a larger portion of the input, as shown in figure. This allows the network to efficiently describe complicated interactions between many variables by constructing such interactions from simple building blocks that each describe only sparse interactions

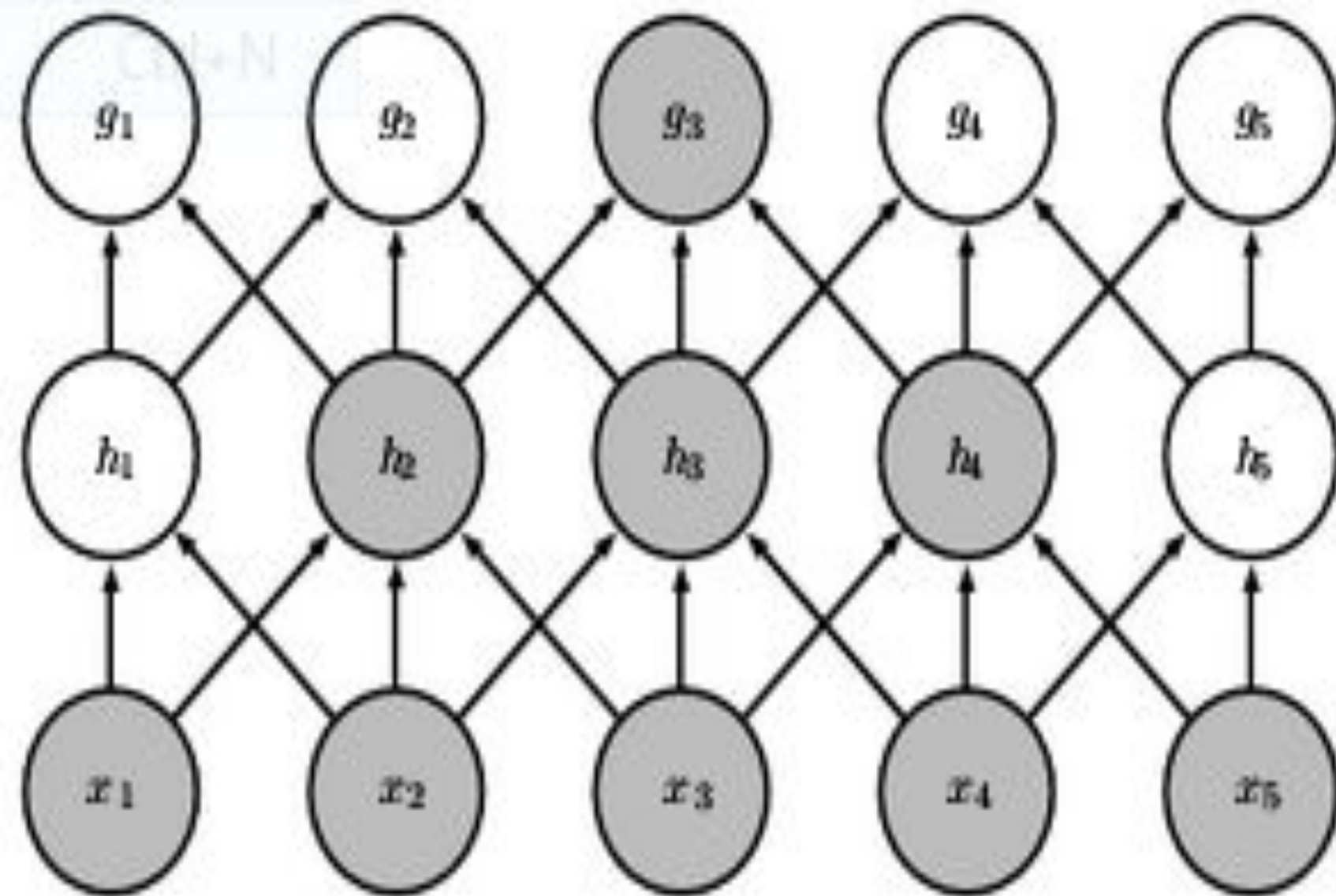


Figure 9.4: The receptive field of the units in the deeper layers of a convolutional network is larger than the receptive field of the units in the shallow layers. This effect increases if the network includes architectural features like strided convolution (figure 9.12) or pooling (section 9.3). This means that even though direct connections in a convolutional net are very sparse, units in the deeper layers can be indirectly connected to all or most of the input image.

Parameter sharing :

Parameter sharing refers to using the same parameter for more than one function in a model. In a traditional neural net, each element of the weight matrix is used exactly once when computing the output of a layer. It is multiplied by one element of the input and then never revisited.

As a synonym for parameter sharing, one can say that a network has tied weights , because the value of the weight applied to one input is tied to the value of a weight applied elsewhere.

The parameter sharing used by the convolution operation means that rather than learning a separate set of parameters for every location, we learn only one set. This does not affect the runtime of forward propagation—it is still $O(k \times n)$ —but it does further reduce the storage requirements of the model to k parameters. k is usually several orders of magnitude smaller than m . Since m and n are usually roughly the same size, k is practically in significant compared to $m \times n$. Convolution is thus dramatically more efficient than dense matrix multiplication in terms of the memory requirements and statistical efficiency. For a graphical depiction of how parameter sharing works, see figure 9.5.

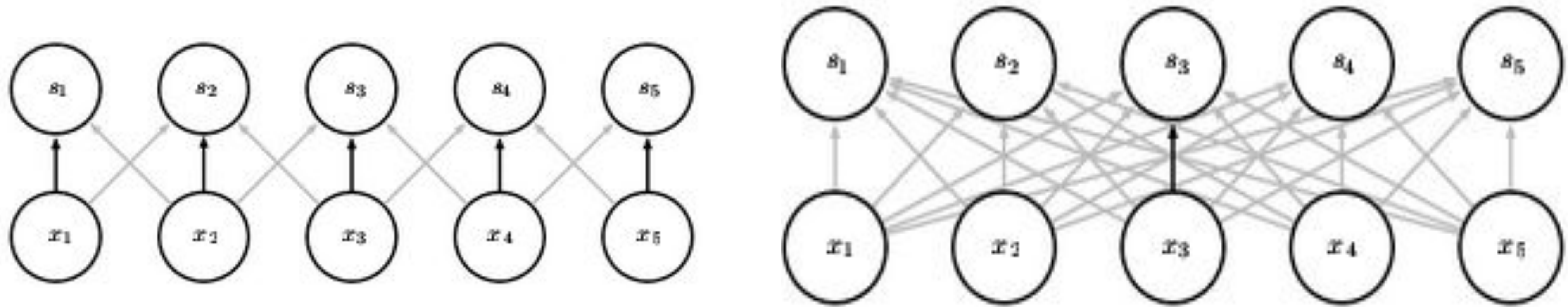


Figure 9.5: Parameter sharing. Black arrows indicate the connections that use a particular parameter in two different models. *(Top)* The black arrows indicate uses of the central element of a 3-element kernel in a convolutional model. Because of parameter sharing, this single parameter is used at all input locations. *(Bottom)* The single black arrow indicates the use of the central element of the weight matrix in a fully connected model. This model has no parameter sharing, so the parameter is used only once.

In the case of convolution, the particular form of parameter sharing causes the layer to have a property called **equivariance** to translation. To say a function is equivariant means that if the input changes, the output changes in the same way. Specifically, a function $f(x)$ is equivariant to a function g if $f(g(x)) = g(f(x))$.

Pooling Layer

- Pooling, also known as subsampling or downsampling, is used in CNNs to reduce the spatial dimensions (width and height) of the input feature maps while retaining the most important information.
- It involves sliding a two-dimensional filter over each channel of a feature map and summarizing the features within the region covered by the filter.
- It helps in controlling the model's complexity, reducing overfitting, and improving computational efficiency by reducing the number of parameters and computation required in subsequent layers.

For a feature map with dimensions $n_h \times n_w \times n_c$, the dimensions of the output after a pooling layer are:

$$\left(\frac{n_h - f + 1}{s} \right) \times \left(\frac{n_w - f + 1}{s} \right) \times n_c$$

where:

- $n_h \rightarrow$ height of the feature map
- $n_w \rightarrow$ width of the feature map
- $n_c \rightarrow$ number of channels in the feature map
- $f \rightarrow$ size of the pooling filter
- $s \rightarrow$ stride length

A typical CNN model architecture consists of multiple convolution and pooling layers stacked together.

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

filter of size 2 by 2 with stride of 1 by 1

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

filter of size 3 by 3 with stride of 1 by 1

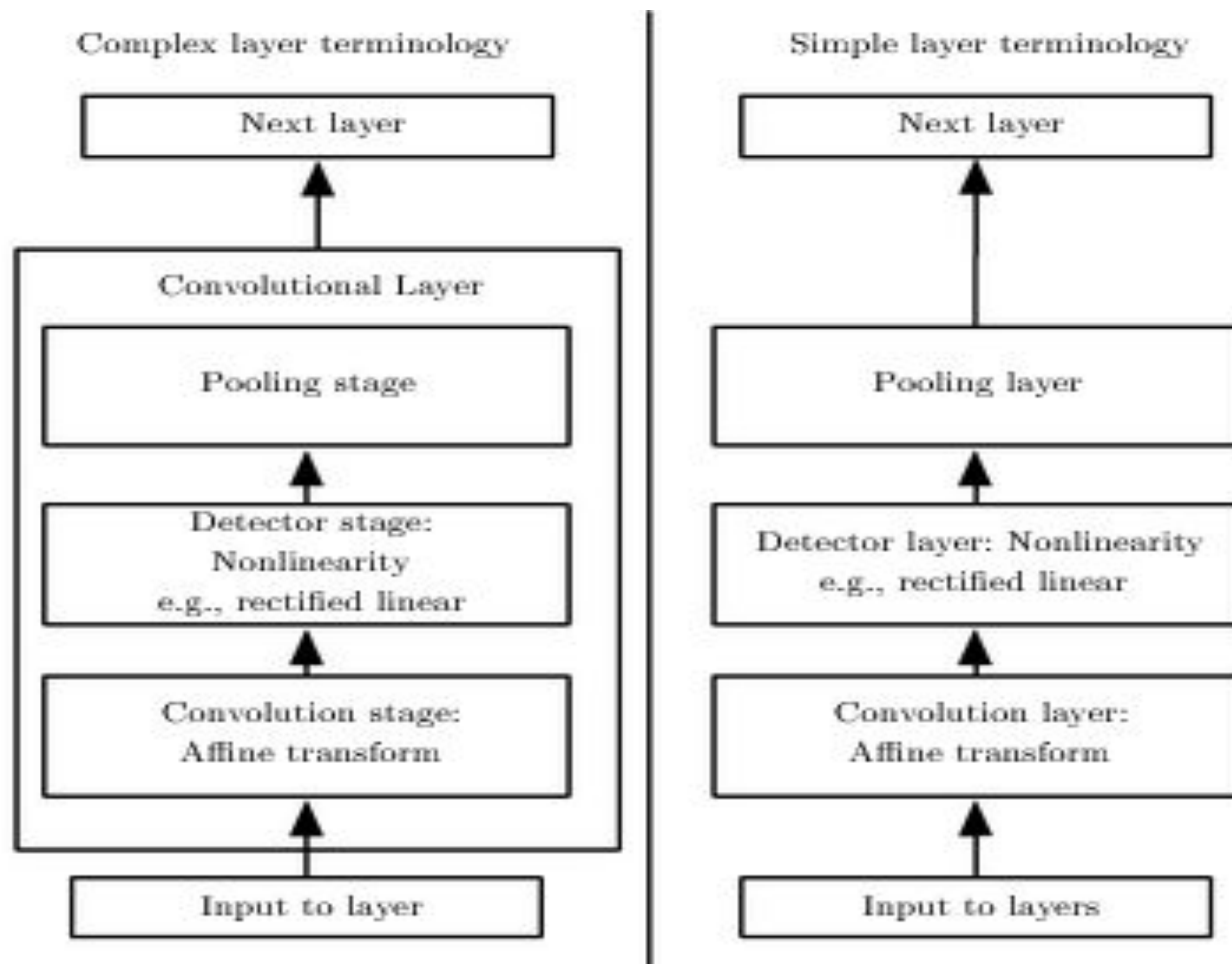
1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

filter of size 3 by 3 with stride of 2 by 2

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

filter of size 2 by 2 with stride of 2 by 2

9.3 Pooling : A pooling function replaces the output of the net at a certain location with a summary statistic of the nearby outputs. For example, the max pooling (Zhou and hellappa, 1988) operation reports the maximum output within a rectangular neighborhood.



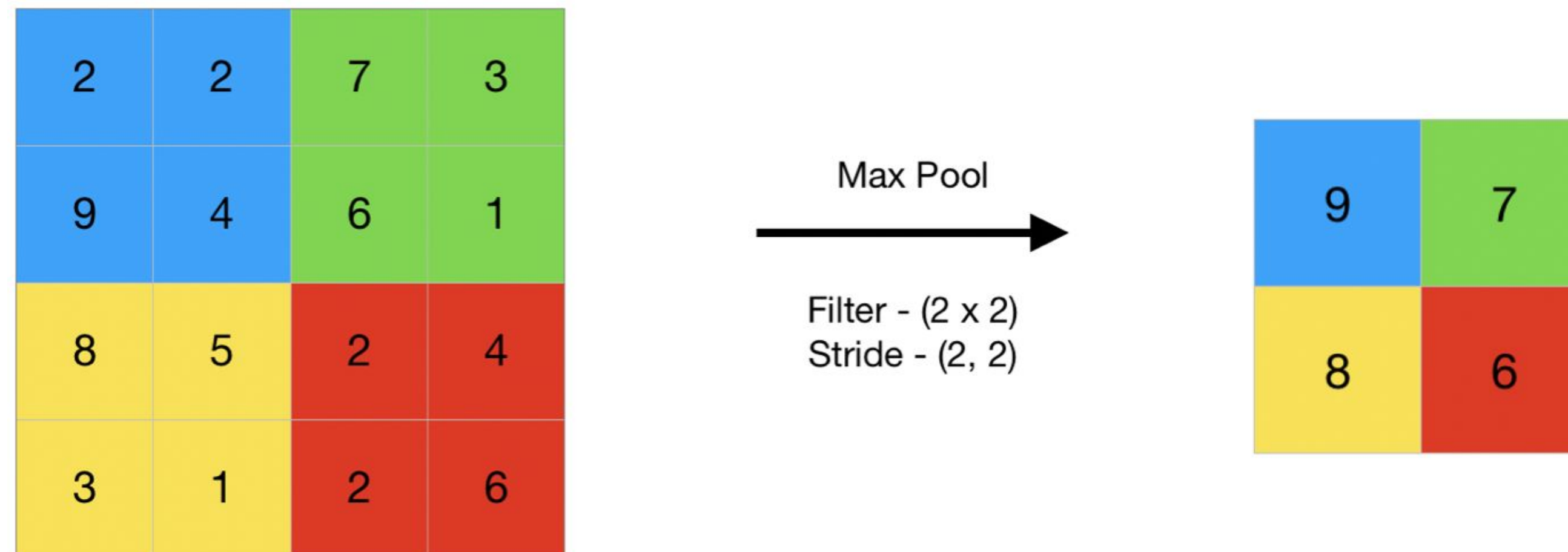
Why are Pooling Layers Important?

- **Dimensionality Reduction:** Pooling layers reduce the spatial size of the feature maps, which decreases the number of parameters and computations in the network. This makes the model faster and more efficient.
- **Translation Invariance:** Pooling helps the network become invariant to small translations or distortions in the input image. For example, even if an object in an image is slightly shifted, the pooled output will remain relatively unchanged.
- **Overfitting Prevention:** By reducing the spatial dimensions, pooling layers help prevent overfitting by providing a form of regularization.
- **Feature Hierarchy:** Pooling layers help build a hierarchical representation of features, where lower layers capture fine details and higher layers capture more abstract and global features.

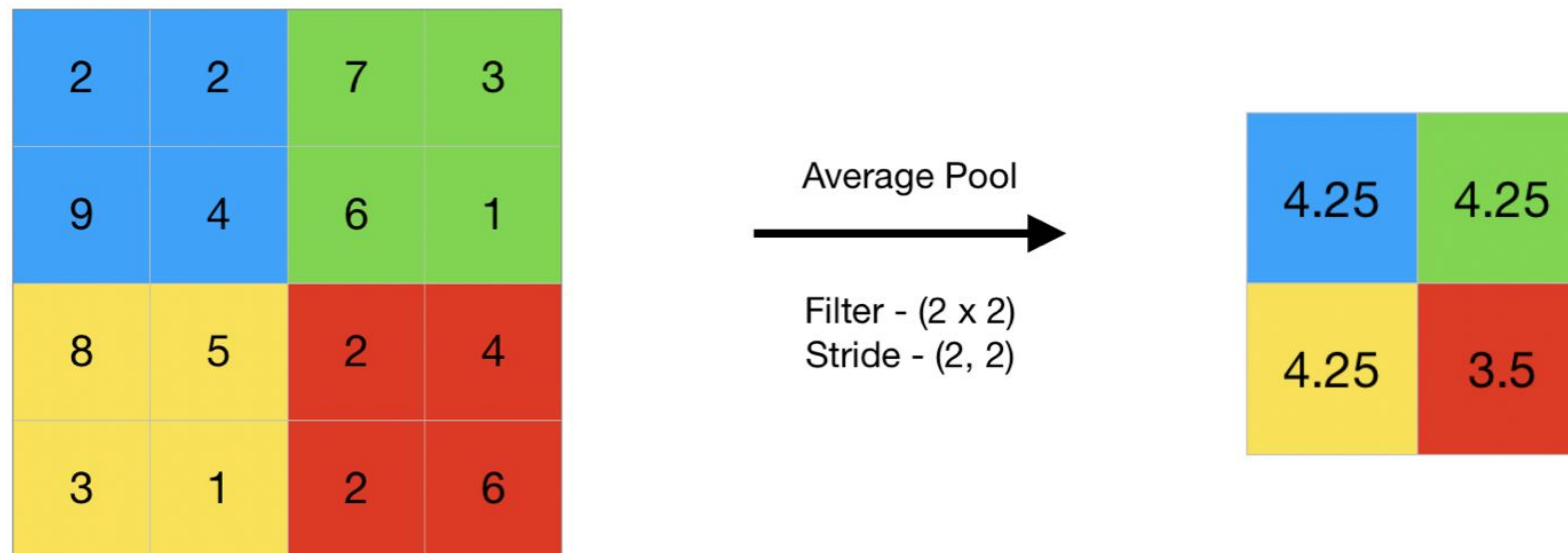
Types of Pooling Layers

Max pooling selects the maximum element from the region of the feature map covered by the filter. Thus, the output after max-pooling layer would be a feature map containing the most prominent features of the previous feature map.

Max pooling layer preserves the most important features (edges, textures, etc.) and provides better performance in most cases.



- **Average pooling** computes the average of the elements present in the region of feature map covered by the filter. Thus, while max pooling gives the most prominent feature in a particular patch of the feature map, average pooling gives the average of features present in a patch.
- Average pooling provides a more generalized representation of the input. It is useful in the cases where preserving the overall context is important.

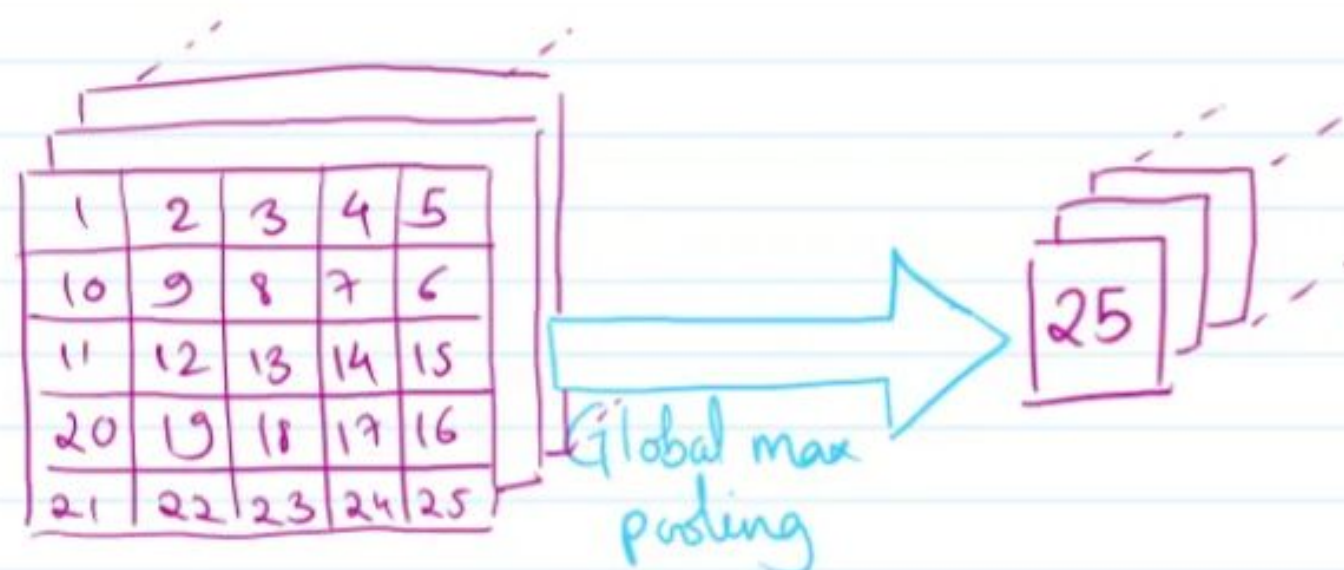


Global pooling reduces each channel in the feature map to a single value, producing a $1 \times 1 \times n_c$ output. This is equivalent to applying a filter of size $n_h \times n_w$.

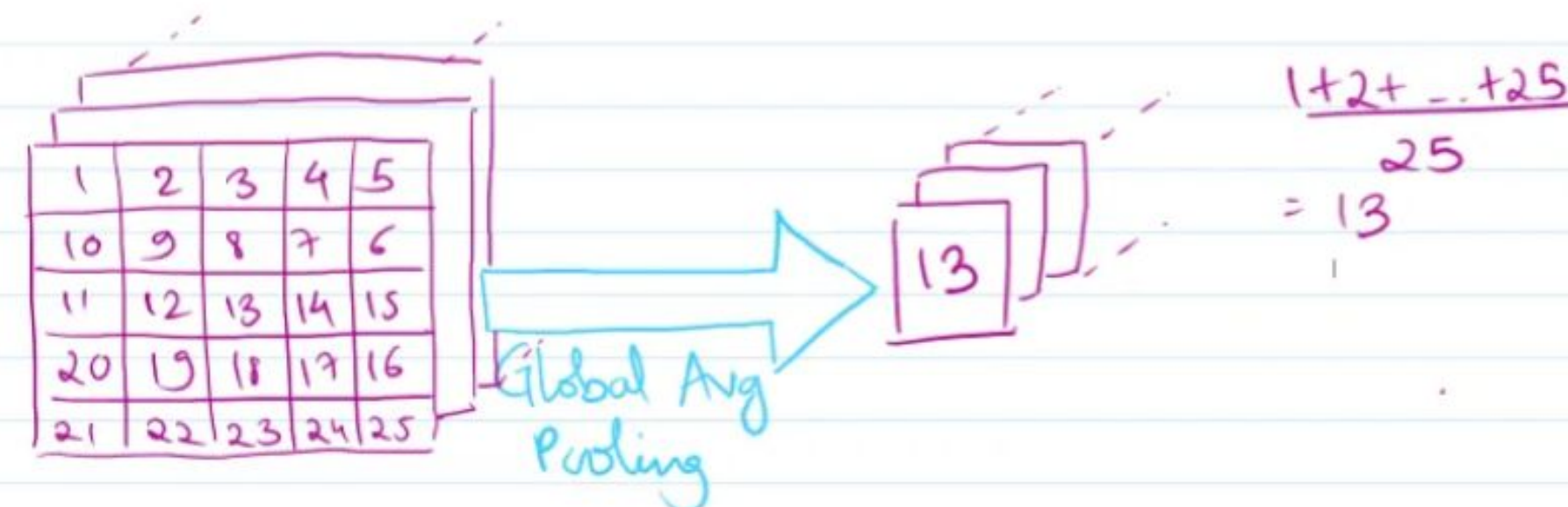
There are two types of global pooling:

- **Global Max Pooling:** Takes the maximum value across the entire feature map.
- **Global Average Pooling:** Computes the average of all values in the feature map.

Global Max Pooling



Global Average Pooling



- In all cases, pooling helps to make the representation become approximately invariant to small translations of the input.
- Invariance to translation means that if we translate the input by a small amount, the values of most of the pooled outputs do not change.

- Invariance to local translation can be a very useful property if we care more about whether some feature is present than exactly where it is.
- For example, when determining whether an image contains a face, we need not know the location of the eyes with pixel-perfect accuracy, we just need to know that there is an eye on the left side of the face and an eye on the right side of the face.
- In other contexts, it is more important to preserve the location of a feature. For example, if we want to find a corner defined by two edges meeting at a specific orientation, we need to preserve the location of the edges well enough to test whether they meet.

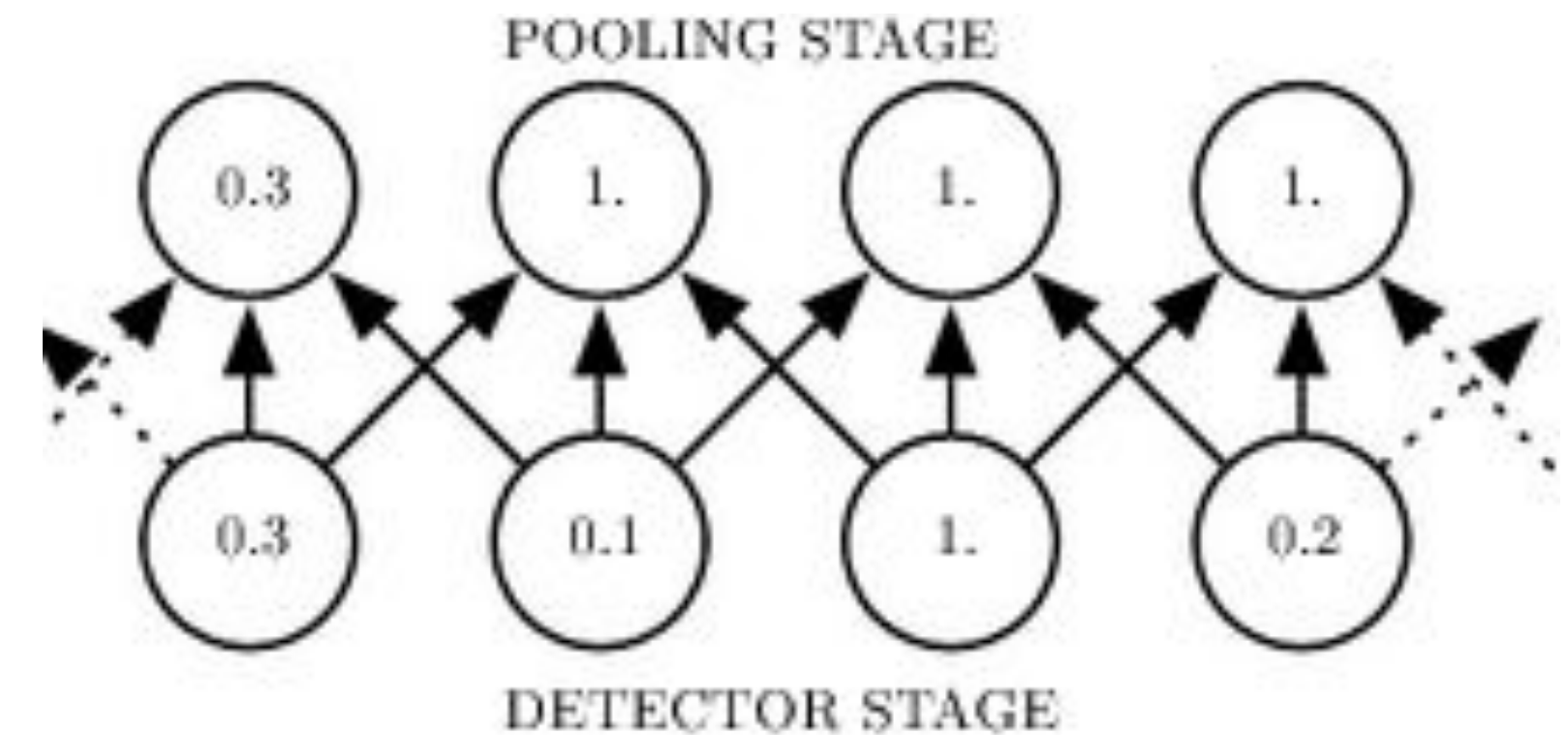
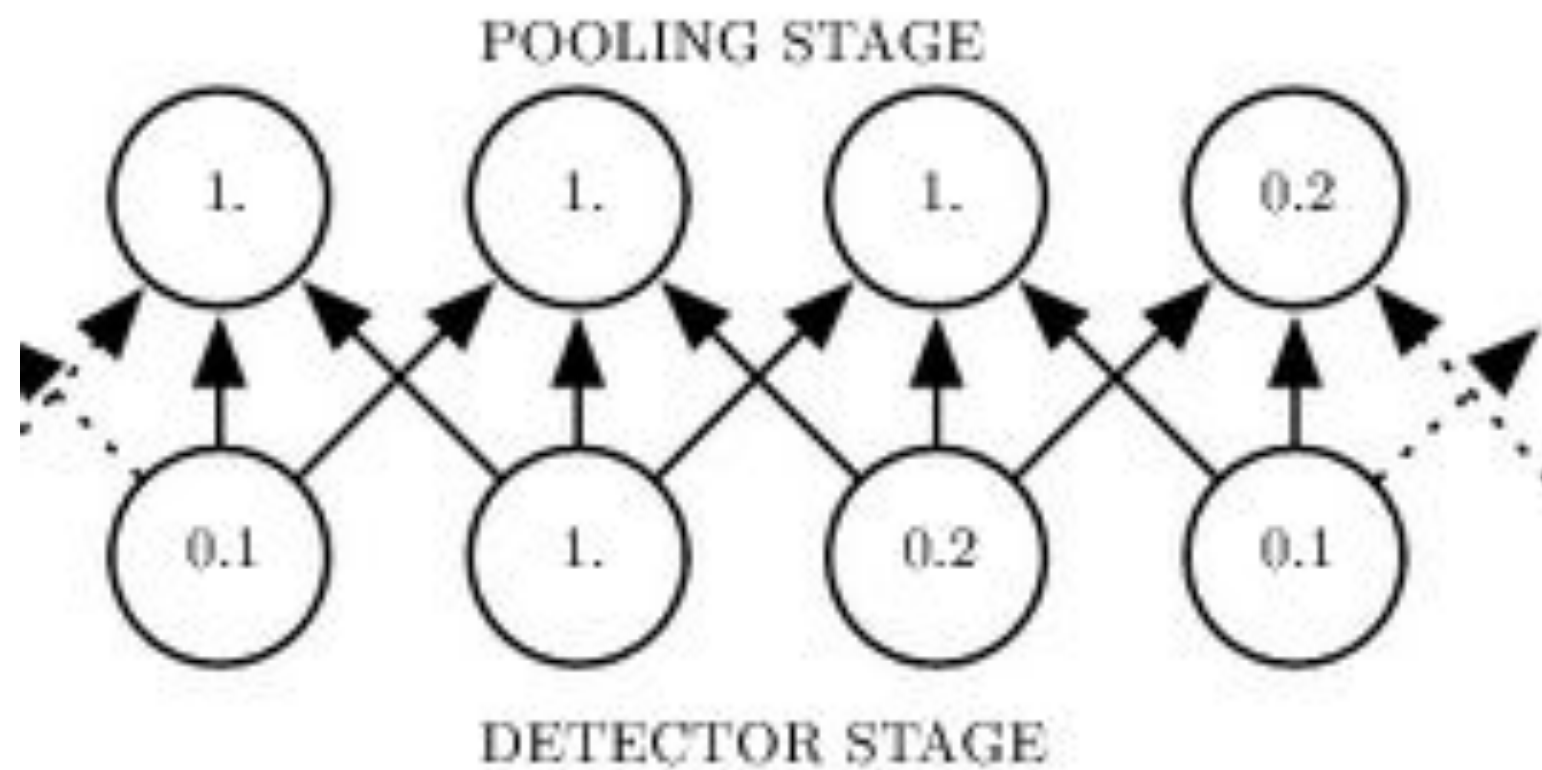
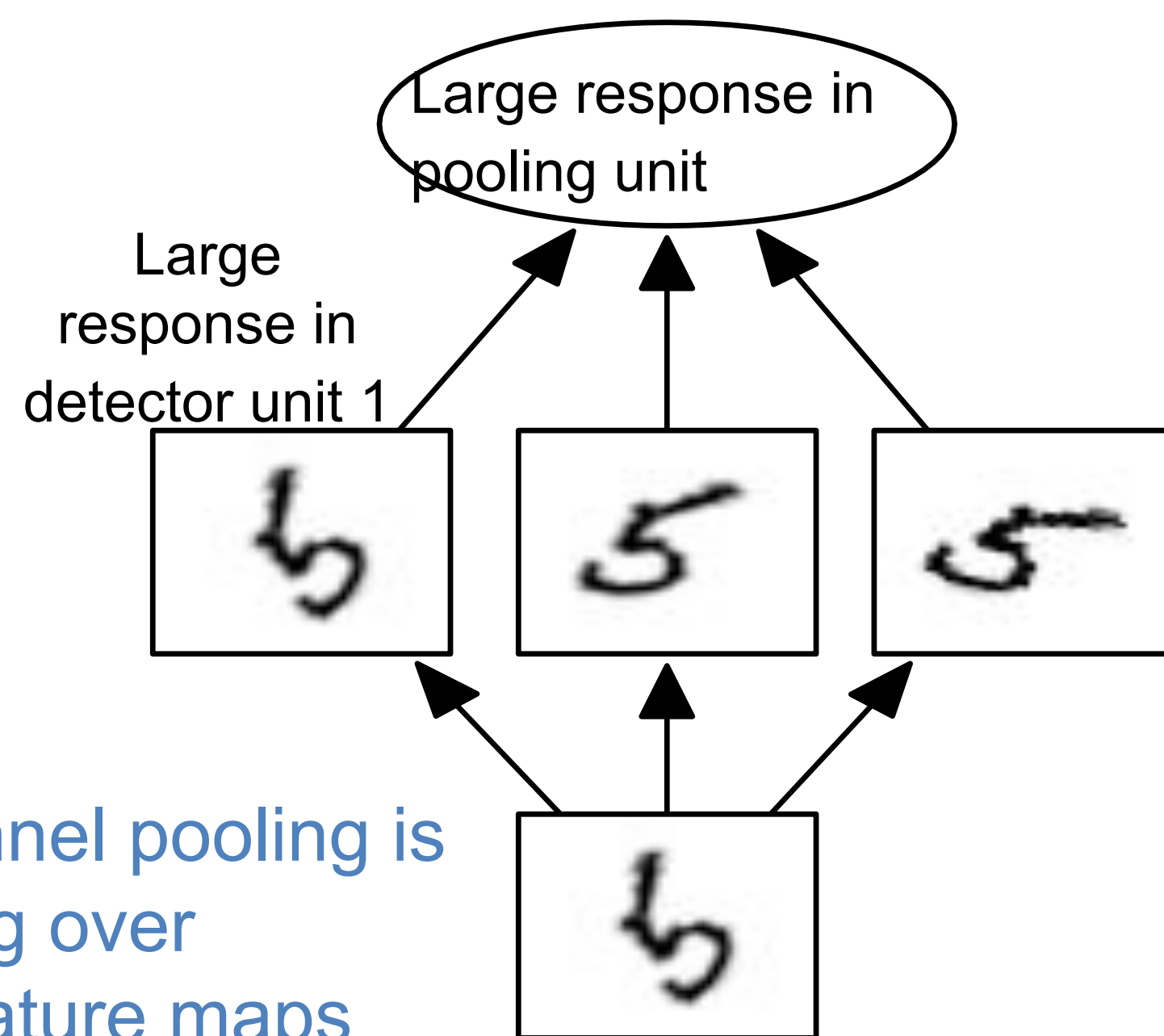
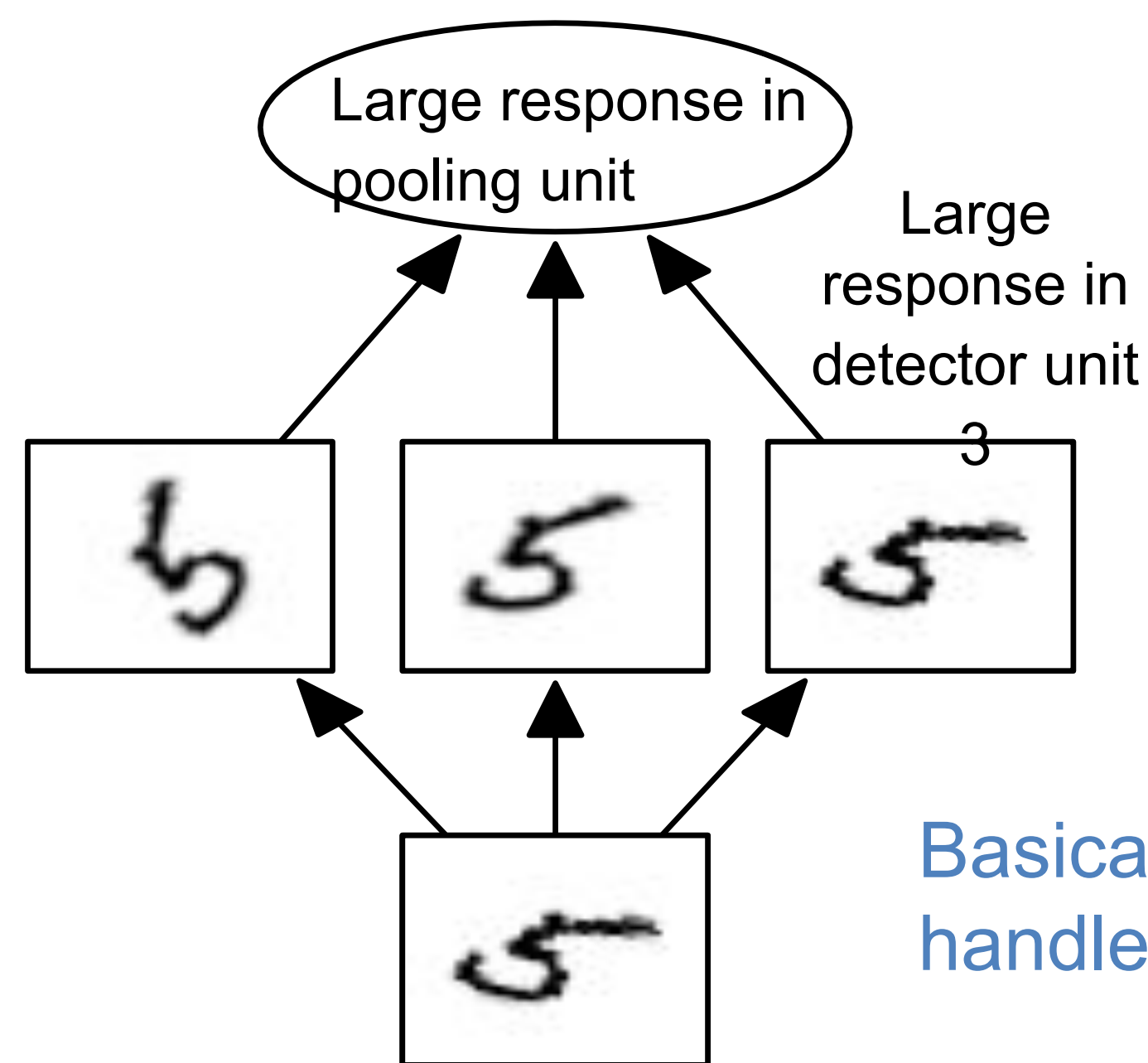


Figure 9.8: Max pooling introduces invariance. *(Top)* A view of the middle of the output of a convolutional layer. The bottom row shows outputs of the nonlinearity. The top row shows the outputs of max pooling, with a stride of one pixel between pooling regions and a pooling region width of three pixels. *(Bottom)* A view of the same network, after the input has been shifted to the right by one pixel. Every value in the bottom row has changed, but only half of the values in the top row have changed, because the max pooling units are sensitive only to the maximum value in the neighborhood, not its exact location.

Cross-Channel Pooling and Invariance to Learned Transformations



Cross-channel pooling is max pooling over different feature maps

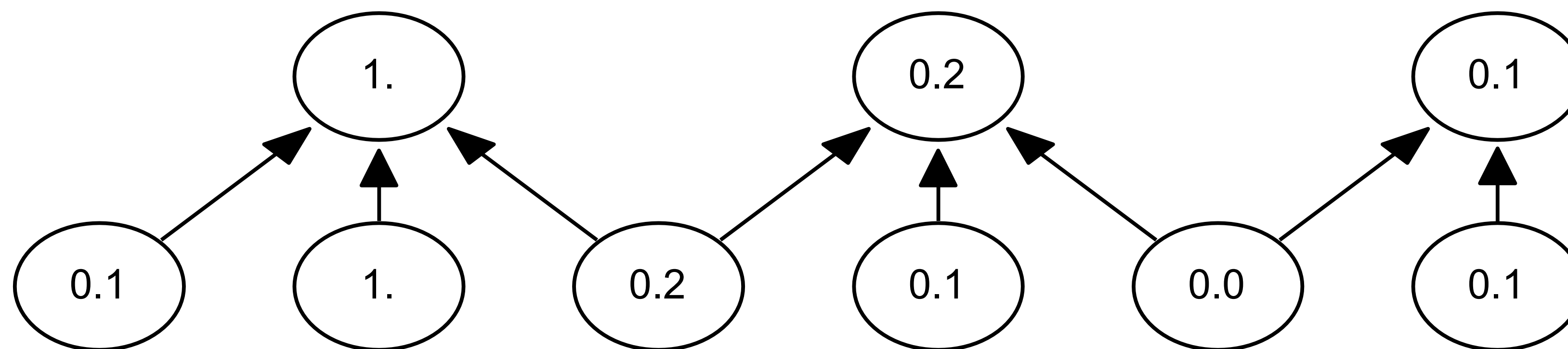


Basically handles rotations

Figure 9.9: Example of learned invariances. A pooling unit that pools over **multiple features that are learned with separate parameters can learn** to be invariant to transformations of the input. Here we show how a set of three learned filters and a max pooling unit can learn to become **invariant to rotation**. All three filters are intended to detect a hand written 5. Each filter attempts to match a slightly different orientation of the 5. When a 5 appears in the input, the corresponding filter will match it and cause a large activation in a detector unit. The max pooling unit then has a large activation regardless of which detector unit was activated. We show here how the network processes two different inputs, resulting in two different detector units being activated. The effect on the pooling unit is roughly the same either way. **This principle is leveraged by maxout networks (Goodfellow et al., 2013a) and other convolutional networks. Max pooling over spatial positions is naturally invariant to translation; this multichannel approach is only necessary for learning other transformations.**

Pooling with Downsampling

Figure 9.10



Max pooling
downsized in next layer

Figure 9.10: Pooling with downsampling. Here we use max pooling with a pool width of three and a stride between pools of two. This reduces the representation size by a factor of two, which reduces the computational and statistical burden on the next layer. Note that the rightmost pooling region has a smaller size but must be included if we do not want to ignore some of the detector units

Example Classification Architectures

Figure
9.11

fixed image size

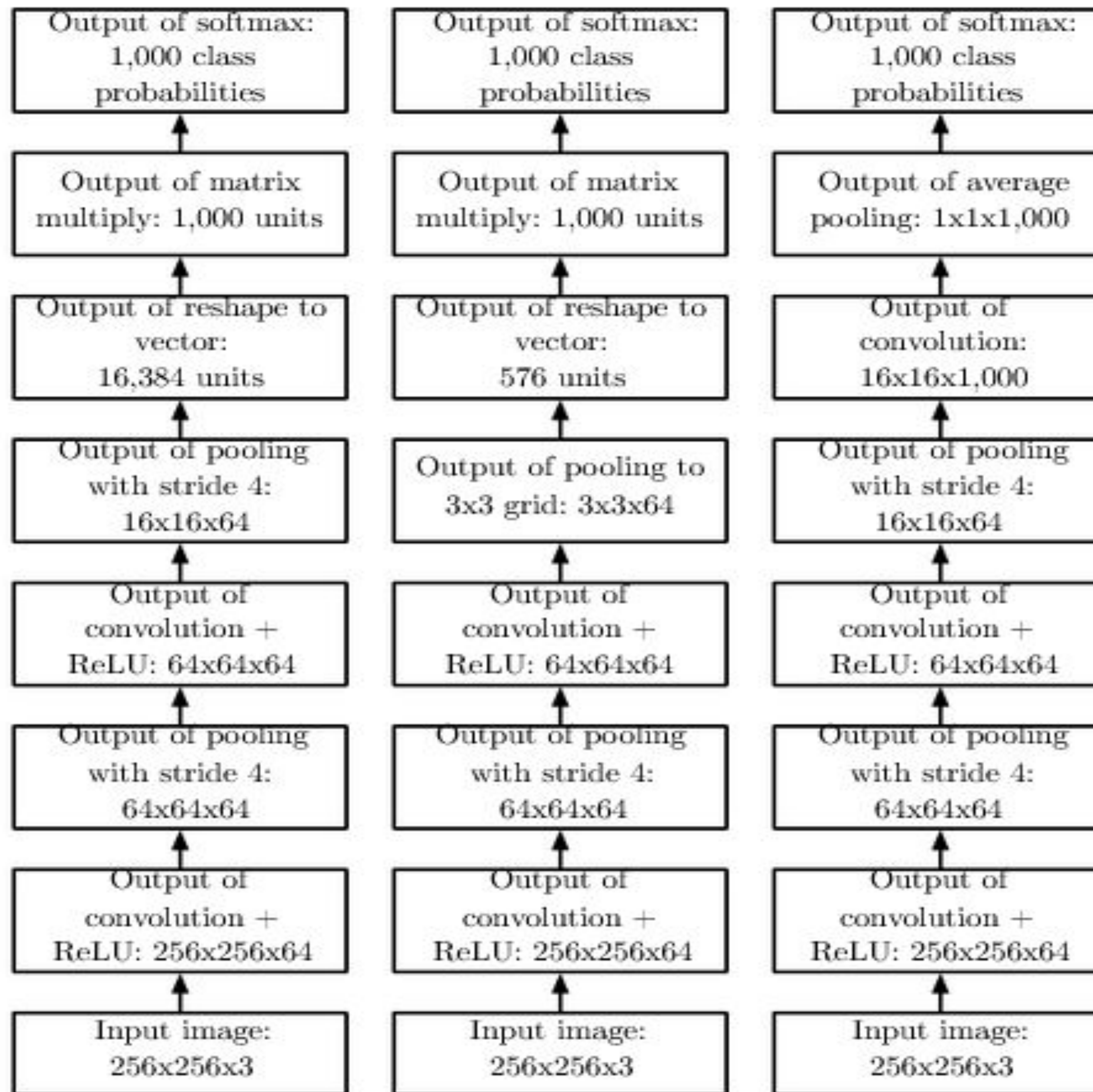


Figure 9.11: Examples of architectures for classification with convolutional networks. The specific strides and depths used in this figure are not advisable for real use; they are designed to be very shallow to fit onto the page. Real convolutional networks also often involve significant amounts of branching, unlike the chain structures used here for simplicity.

(Left) A convolutional network that processes a fixed image size. After alternating between convolution and pooling for a few layers, the tensor for the convolutional feature map is reshaped to flatten out the spatial dimensions. The rest of the network is an ordinary feedforward network classifier, as described in chapter 6.

(Center) A convolutional network that processes a variably sized image but still maintains a fully connected section. This network uses a pooling operation with variably sized pools but a fixed number of pools, in order to provide a fixed-size vector of 576 units to the fully connected portion of the network. (Right) A convolutional network that does not have any fully connected weight layer. Instead, the last convolutional layer outputs one feature map per class. The model presumably learns a map of how likely each class is to occur at each spatial location. Averaging a feature map down to a single value provides the argument to the softmax classifier at the top

4. Convolution and Pooling as an Infinitely Strong Prior

- Prior probabilities (beliefs before we see actual data) can be strong or weak
 - A weak prior (e.g., Gaussian distribution with high variance) allows the data to move the parameters
 - A strong prior (e.g., Gaussian distribution with low variance) strongly determines the parameters
 - An **infinitely strong prior** controls the parameters
- A convolutional net $\sim$ an infinitely strong prior
 - Weights are zero except in small receptive fields
 - Weights identical for neighboring hidden units

4. Convolution and Pooling as an Infinitely Strong Prior

- Convolution and pooling can cause underfitting
 - The prior is useful only when the assumptions made by the prior are reasonably accurate
 - If a task relies on preserving precise spatial information, then pooling can increase training error
 - The prior imposed by convolution must be appropriate

5. Variants of the Basic Convolution Function

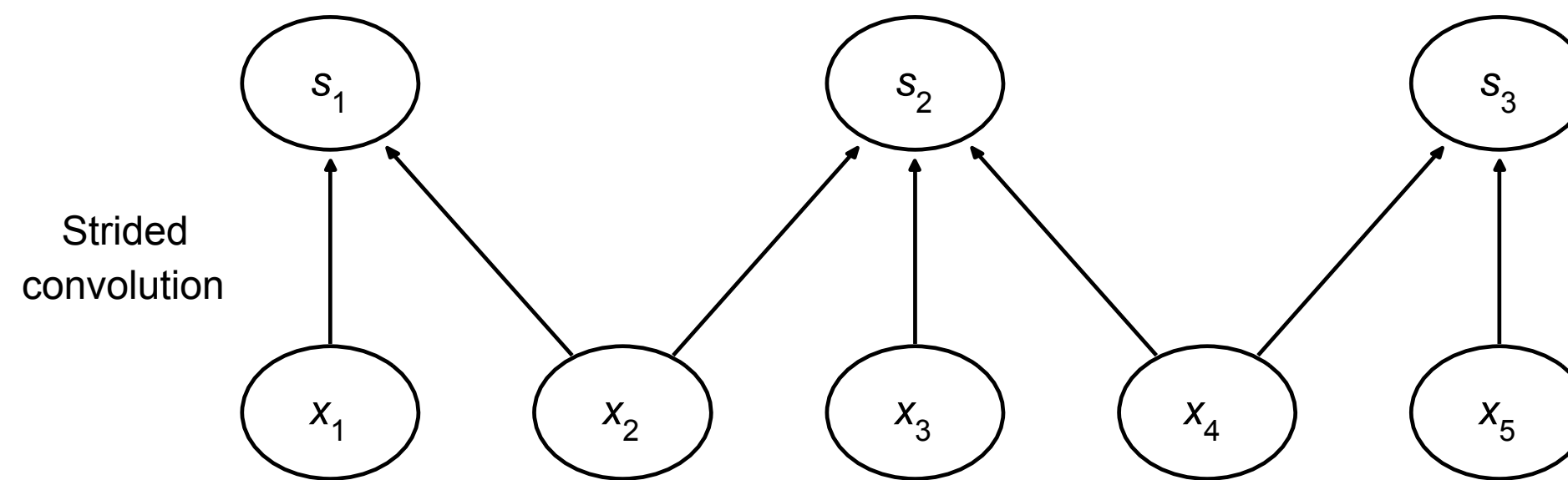
- **Stride** is the amount of downsampling
 - Can have separate strides in different directions
- **Zero padding** avoids layer-to-layer shrinking
- **Unshared convolution**
 - Like convolution but without sharing
- **Partial connectivity between channels**
- **Tiled convolution**
 - Cycle between shared parameter groups

Figure 9.12: Convolution with a stride. In this example, we use a stride of two.
(Top) Convolution with a stride length of two implemented in a single operation.

(Bottom) Convolution with a stride greater than one pixel is mathematically equivalent to convolution with unit stride followed by downsampling. Obviously, the two-step approach involving downsampling is computationally wasteful, because it computes many values that are then discarded.

Convolution with Stride

Stride of two



Equivalent to above but
computationally wasteful

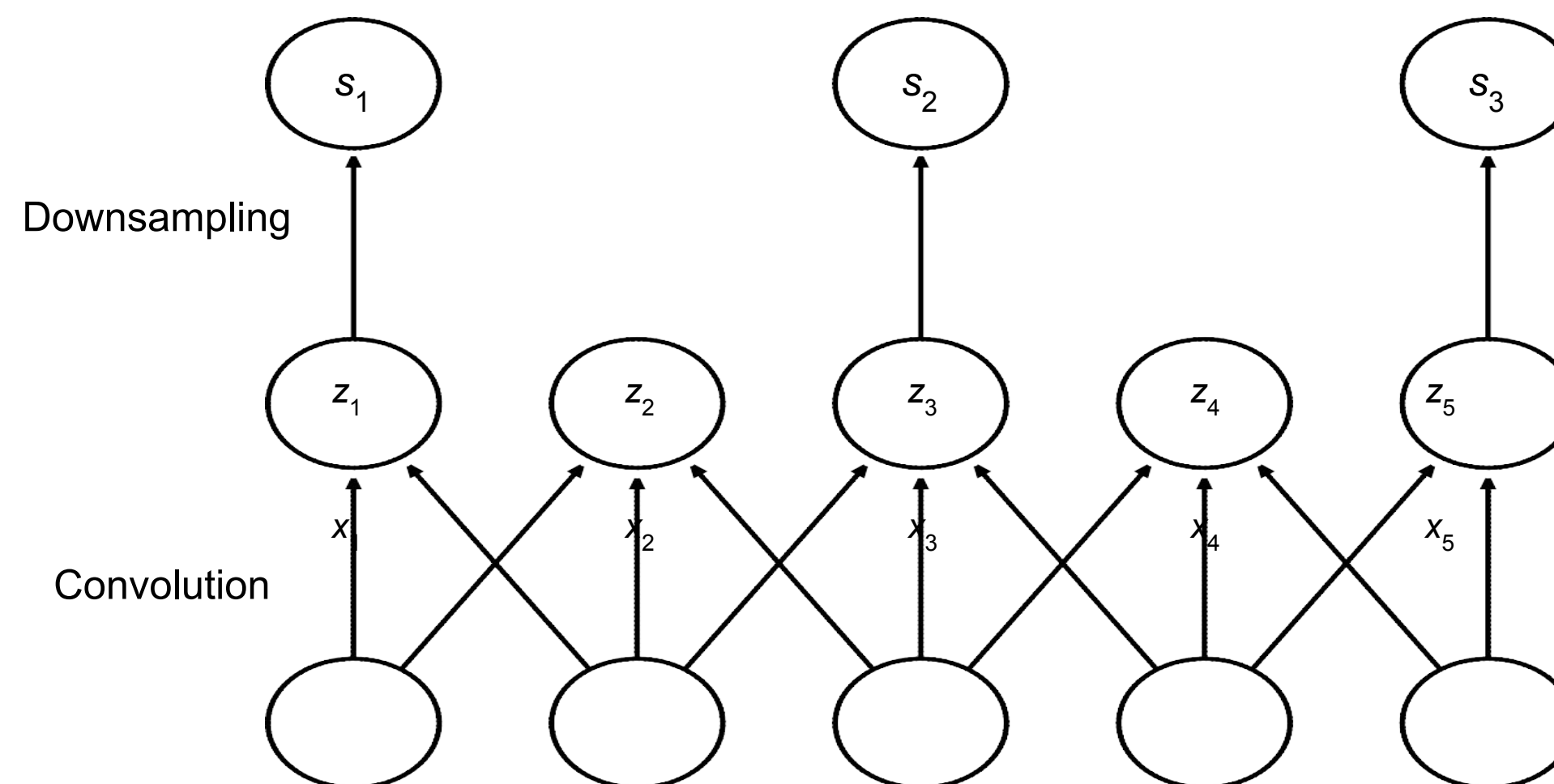
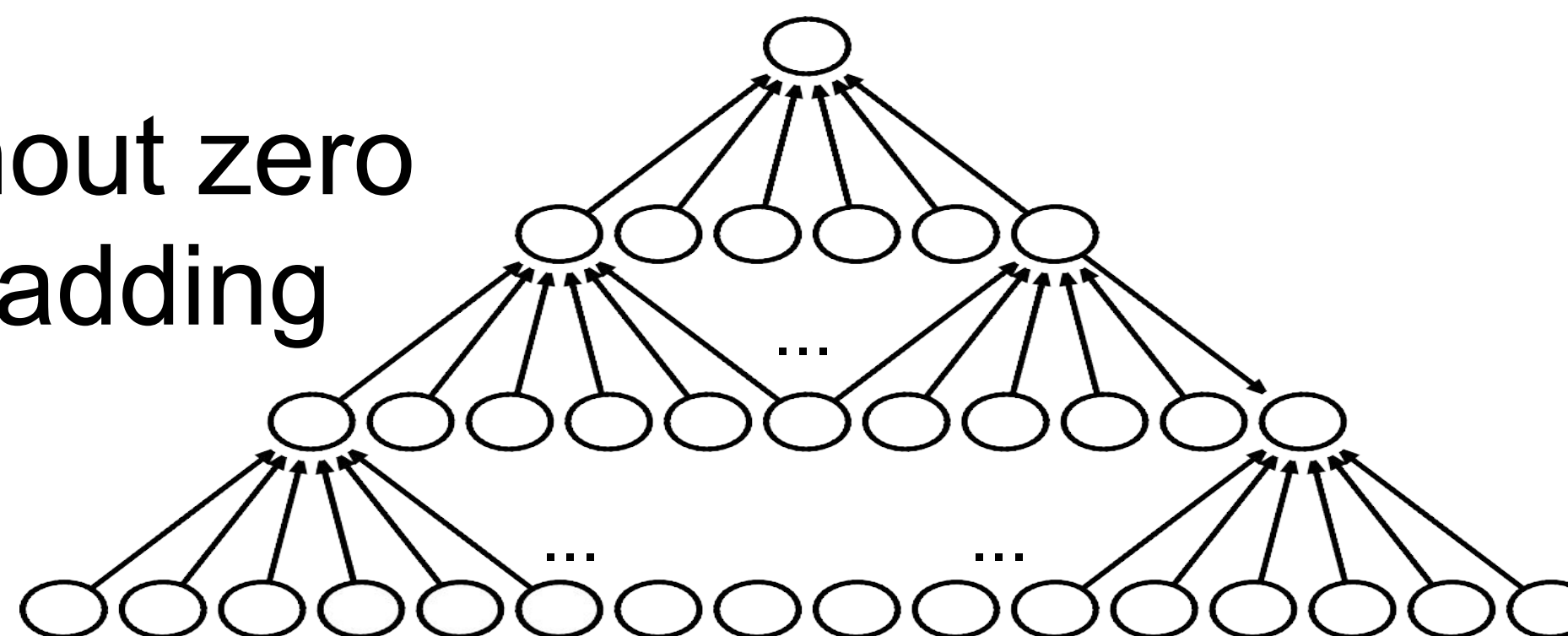


Figure 9.12

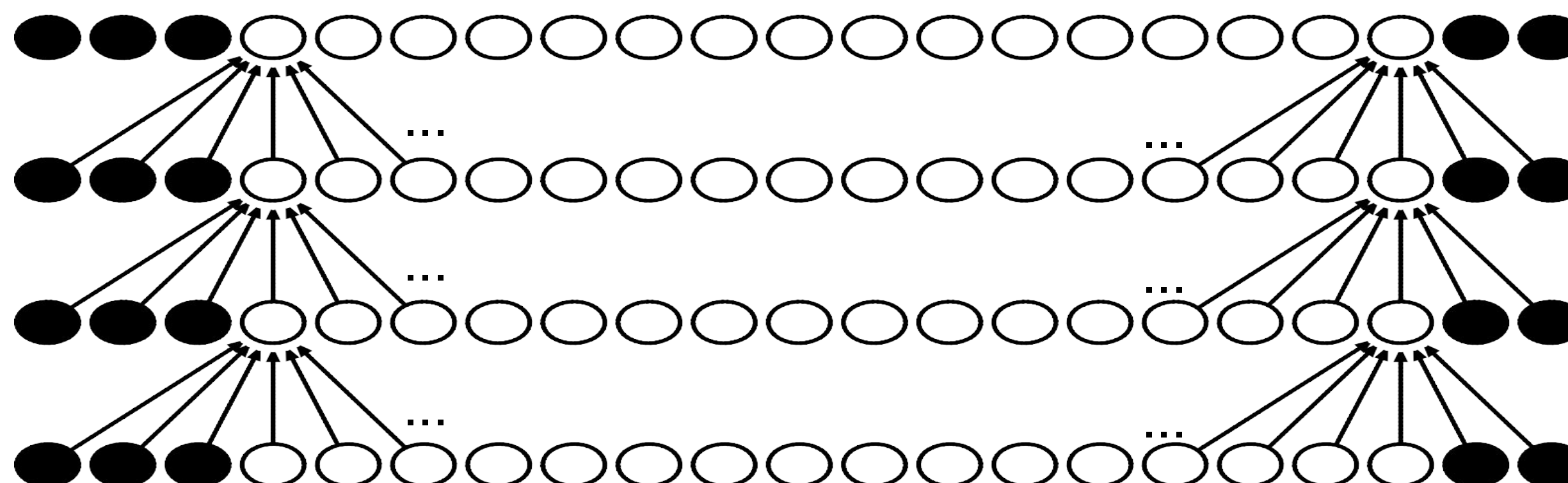
Zero Padding Controls Size

Kernel
width of six

Without zero
padding



With zero
padding
Prevents
shrinking



(Goodfellow 2016)

Figure 9.13

Figure 9.14: Comparison of local connections, convolution, and full connections.

(Top) A locally connected layer with a patch size of two pixels. Each edge is labeled with a unique letter to show that each edge is associated with its own weight parameter.

(Center) A convolutional layer with a kernel width of two pixels. This model has exactly the same connectivity as the locally connected layer. The difference lies not in which units interact with each other, but in how the parameters are shared. The locally connected layer

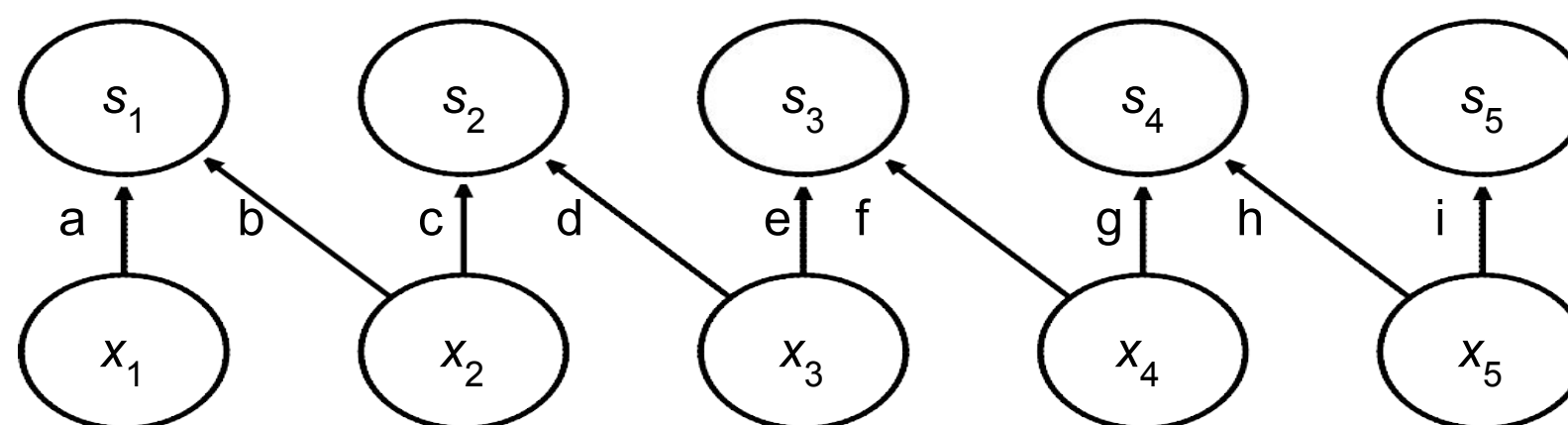
has no parameter sharing. The convolutional layer uses the same two weights repeatedly across the entire input, as indicated by the repetition of the letters labeling each edge.

(Bottom) A fully connected layer resembles a locally connected layer in the sense that each edge has its own parameter (there are too many to label explicitly with letters in this diagram). It does not, however, have the restricted connectivity of the locally connected layer

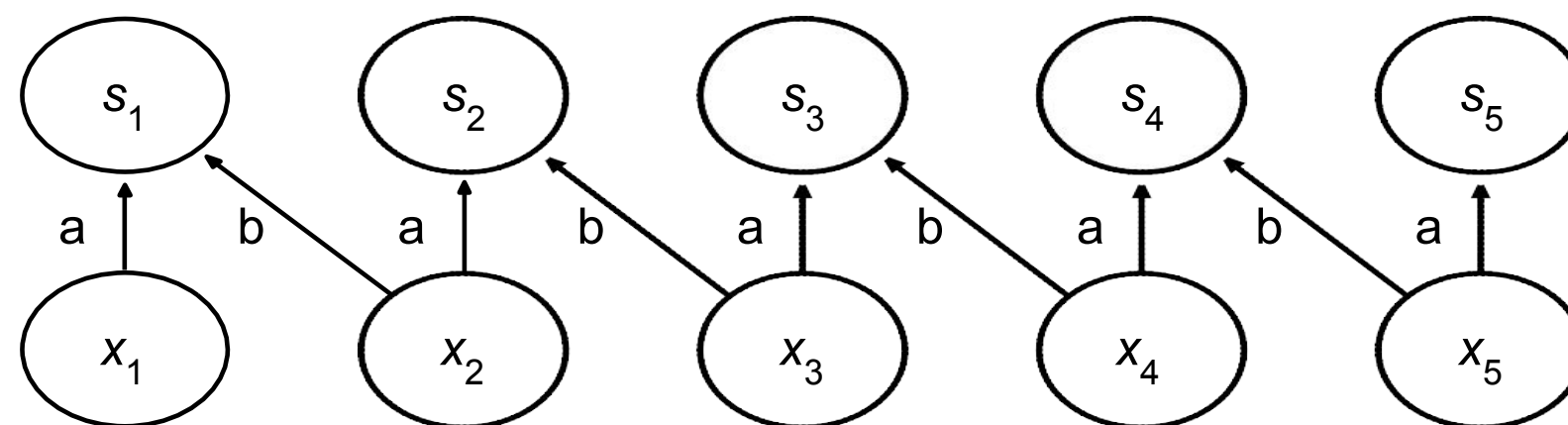
Kinds of Connectivity

Go, change the world

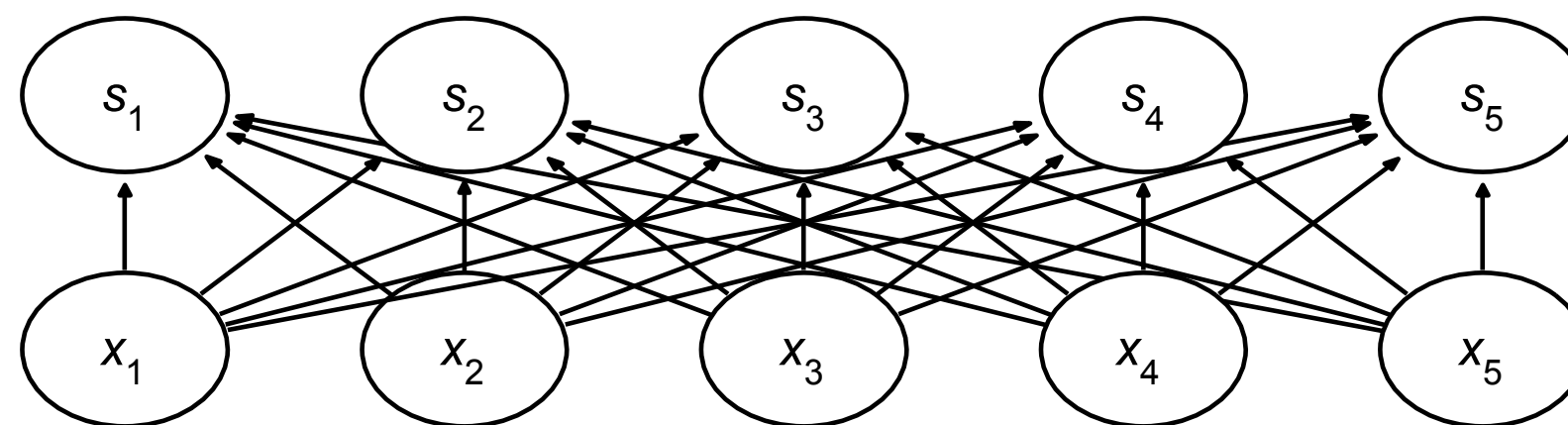
Unshared
convolution



Local connection:
like convolution, but
no sharing



Convolution



Fully connected

Figure 9.14

Locally connected layers are useful when we know that each feature should be a function of a small part of space, but there is no reason to think that the same feature should occur across all of space. **For example, if we want to tell if an image is a picture of a face, we only need to look for the mouth in the bottom half of the image.** It can also be useful to make versions of convolution or locally connected layers in which the connectivity is further restricted, for example to constrain each output channel i to be a function of only a subset of the input channels I . A common way to do this is to make the first m output channels connect to only the first n input channels, the second m output channels connect to only the second n input channels, and so on. See figure 9.15

Partial Connectivity Between Channels

Each output channel
is a function of only a
subset of the input
channels

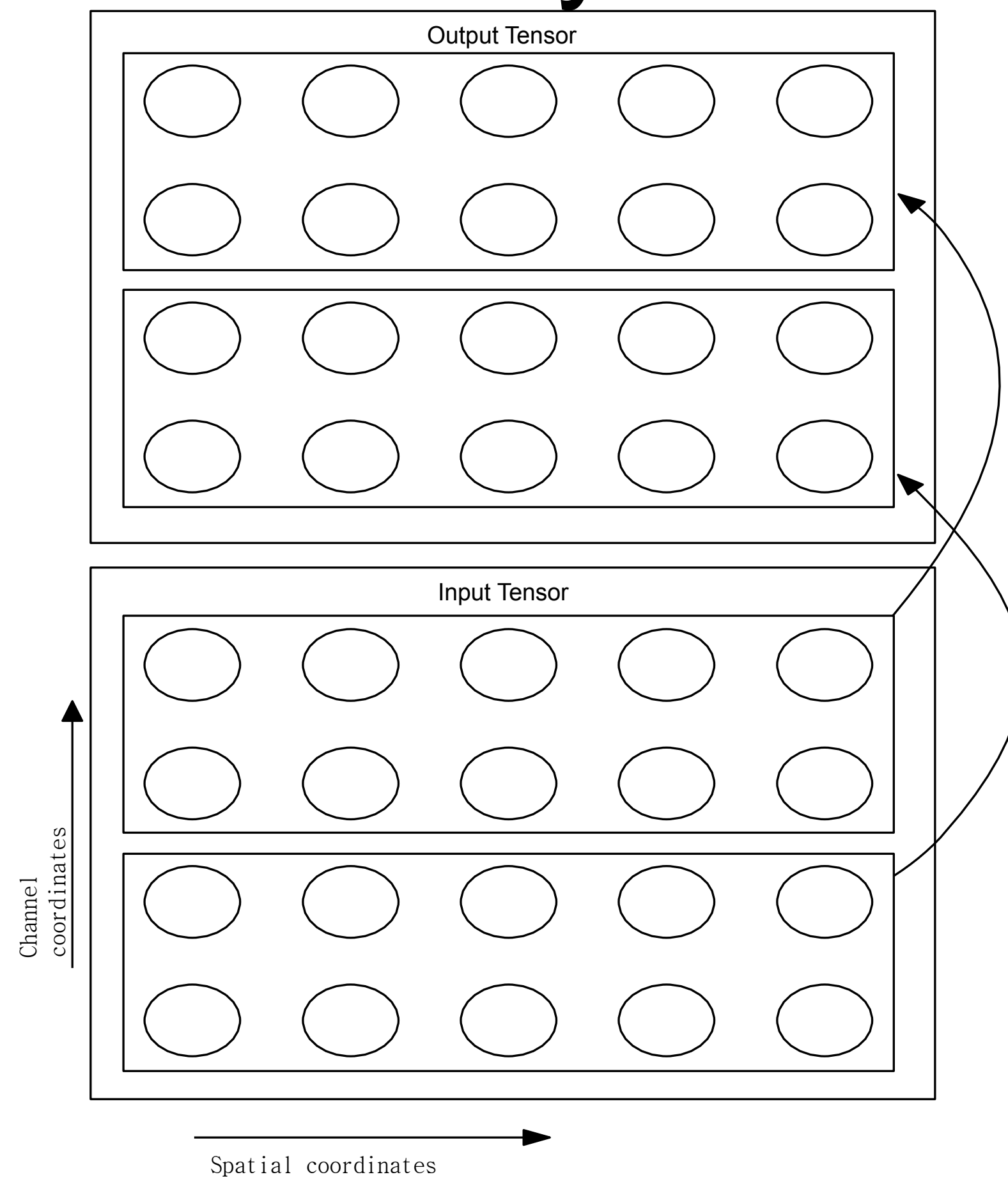
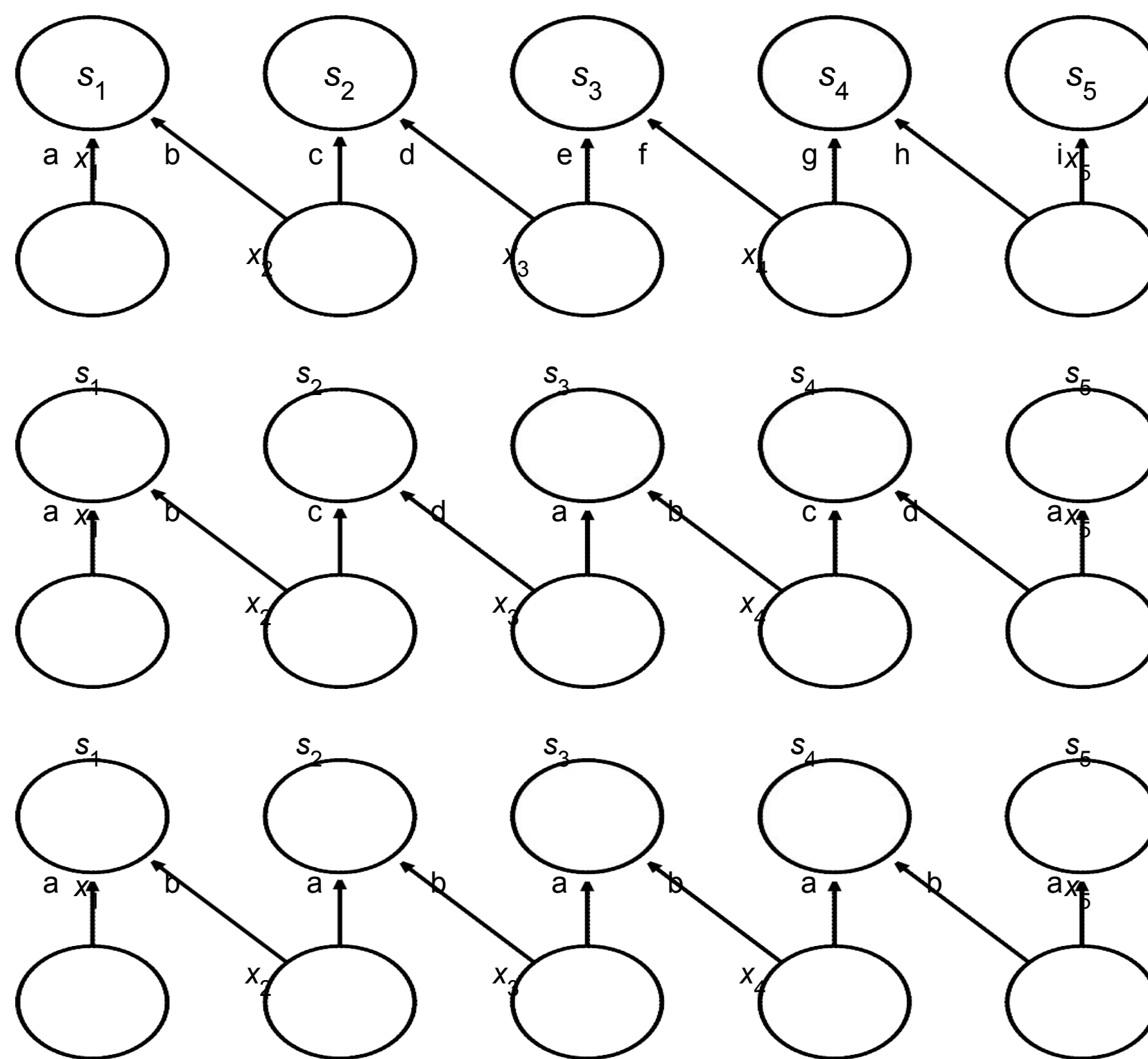


Figure 9.15

Tiled convolution

Go, change the world



Local connection (no sharing)

Tiled convolution (cycle between
shared parameter groups)

Standard convolution
(one group shared everywhere)

Figure 9.16: A comparison of locally connected layers, tiled convolution, and standard convolution. All three have the same sets of connections between units, when the same size of kernel is used.

This diagram illustrates the use of a kernel that is two pixels wide. The differences between the methods lies in how they share parameters. (Top) A locally connected layer has no sharing at all. We indicate that each connection has its own weight by labeling each connection with a unique letter. (Center) Tiled convolution has a set of t different kernels. Here we illustrate the case of $t = 2$.

One of these kernels has edges labeled “a” and “b,” while the other has edges labeled “c” and “d.” Each time we move one pixel to the right in the output, we move on to using a different kernel. This means that, like the locally connected layer, neighboring units in the output have different parameters. Unlike the locally connected layer, after we have gone through all t available kernels, we cycle back to the first kernel. If two output units are separated by a multiple of t steps, then they share parameters. (Bottom) Traditional convolution is equivalent to tiled convolution with $t = 1$. There is only one kernel, and it is applied everywhere, as indicated in the diagram by using the kernel with weights labeled “a” and “b” everywhere.

6. Structured Outputs

- Convolutional networks are usually used for classification
- They can also be used to output a high-dimensional, structured object
- The object is typically a tensor For example, the model might emit a tensor S , where $S_{i,j,k}$ is the probability that pixel (j, k) of the input to the network belongs to class i . This allows the model to label every pixel in an image and draw precise masks that follow the outlines of individual objects

One strategy for pixel-wise labeling of images is to produce an initial guess of the image labels, then refine this initial guess using the interactions between neighboring pixels. Repeating this refinement step several times corresponds to using the same convolutions at each stage, sharing weights between the last layers of the deep net

Figure 9.17 shows the architecture of such a recurrent convolutional network

Figure 9.17: An example of a recurrent convolutional network for pixel labeling. The input is an image tensor X , with axes corresponding to image rows, image columns, and channels (red, green, blue)

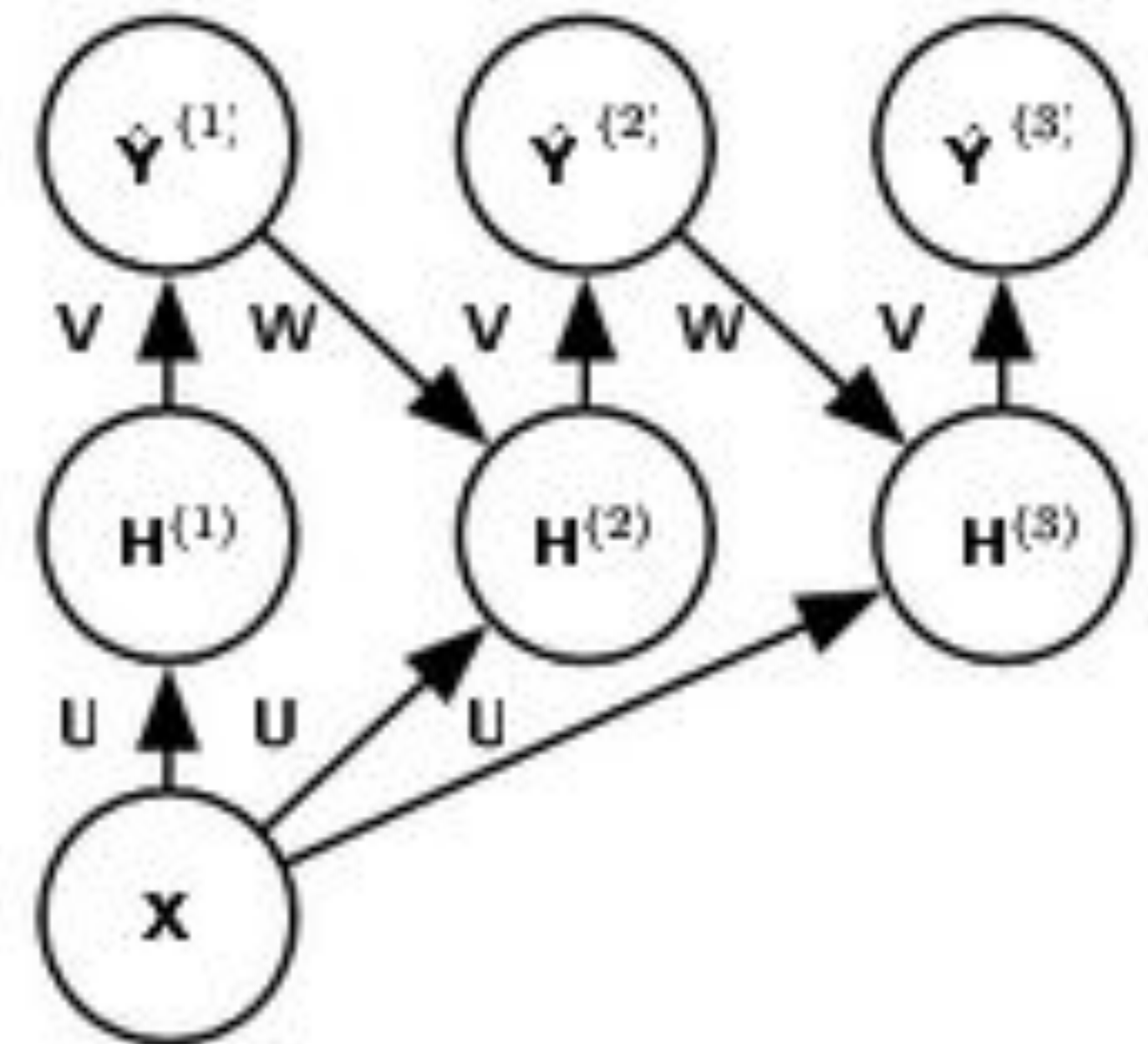


Figure 9.17: An example of a recurrent convolutional network for pixel labeling. The input is an image tensor X , with axes corresponding to image rows, image columns, and channels (red, green, blue)

The goal is to output a tensor of labels $\hat{Y}$, with a probability distribution over labels for each pixel

Rather than outputting $\hat{Y}$ in a single shot, the recurrent network iteratively refines its estimate $\hat{Y}$ by using a previous estimate of $\hat{Y}$ as input for creating a new estimate.

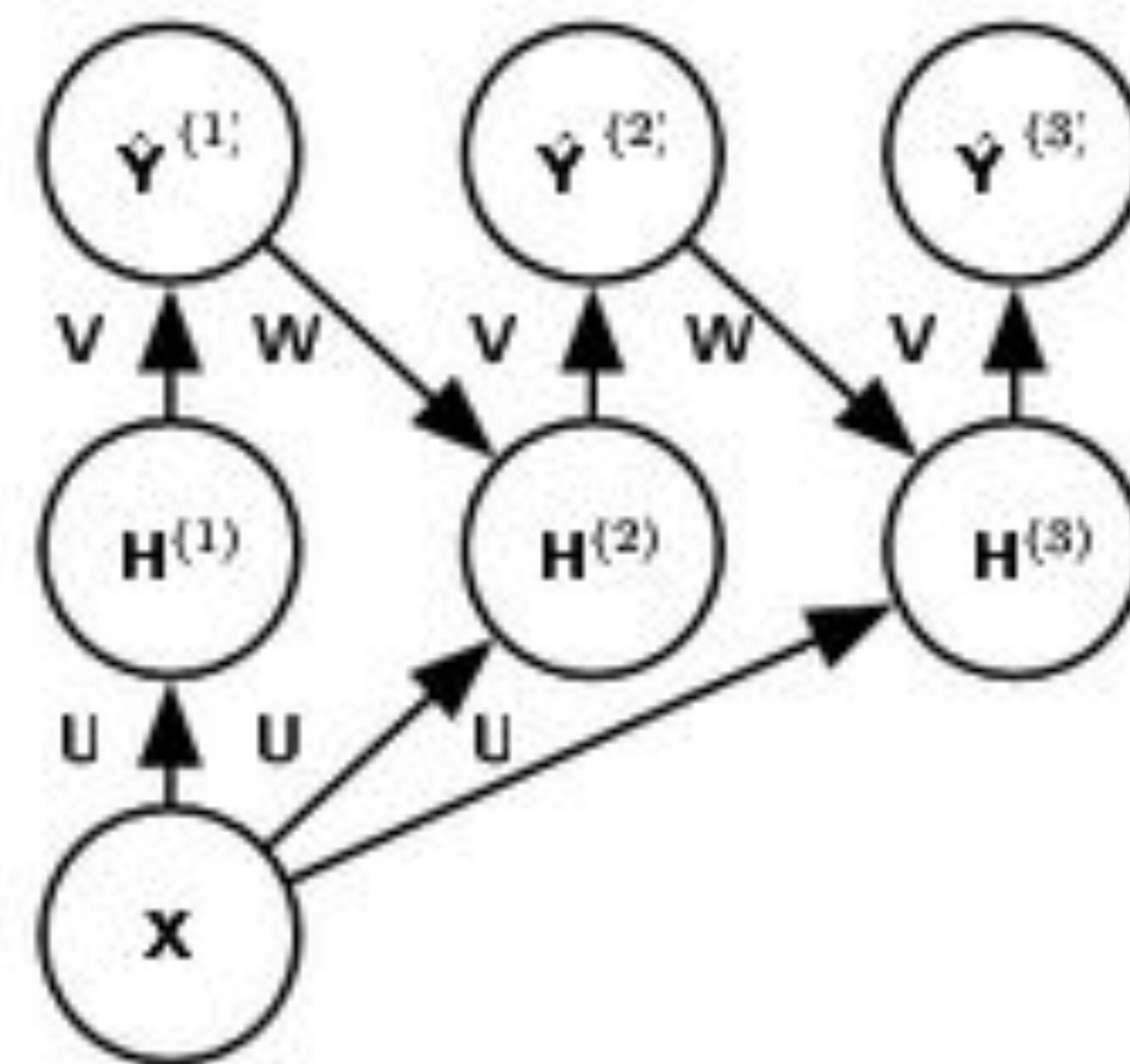
The tensor of convolution kernels U is used on each step to compute the hidden representation given the input image.

The kernel tensor V is used to produce an estimate of the labels given the hidden values.

On all but the first step, the kernels W are convolved over $\hat{Y}$ to provide input to the hidden layer.

On the first time step, this term is replaced by zero.

Because the same parameters are used on each step, this is an example of a recurrent network



7. Data Types

- The data used with a convolutional network usually consist of several channels, each channel being the observation of a different quantity at some point in space or time.
- Single channel examples:
 - 1D audio waveform
 - 2D audio data after Fourier transform
 - Frequency versus time
- Multi-channel examples:
 - 2D color image data
 - Three channels: red pixels, green pixels, blue pixels
 - Each channel is 2D for the image

8. Efficient Convolution Algorithms

- Modern convolutional network applications often involve networks containing more than **one million units**. Powerful implementations exploiting parallel computation resources.
- In many cases, however, it is also possible to speed up convolution by selecting an appropriate convolution algorithm.
- When **a d -dimensional** kernel can be expressed as the outer product of **d vectors**, one vector per dimension, the kernel is called **separable**. When the kernel is separable, naive convolution is inefficient. It is equivalent to compose d one-dimensional convolutions with each of these vectors. The composed approach is significantly faster than performing one d
- -dimensional convolution with their outer product. The kernel also takes fewer parameters to represent as vectors.
- If the kernel is w elements wide in each dimension, then naive multidimensional convolution requires $O(w d)$ runtime and parameter storage space, while separable convolution requires $O(w \times d)$ runtime and parameter storage space. Of course, not every convolution can be represented in this way

8. Efficient Convolution Algorithms

- Devising faster ways of performing convolution or approximate convolution without harming the accuracy of the model is an active area of research
- However, most dissertation work concerns feasibility and not efficiency

9. Random or Unsupervised Features

- Typically, the most expensive part of convolutional network training is learning the features.
- **The output layer is usually relatively inexpensive** because of the small number of features provided as input to this layer after passing through several layers of pooling.
- When performing supervised training with gradient descent, every gradient step requires a complete run of forward propagation and backward propagation through the entire network. **One way to reduce the cost of convolutional network training is to use features that are not trained in a supervised fashion**

9. Random or Unsupervised Features

- One way to reduce the cost of convolutional network training is to use features that are not trained in a supervised fashion
- Three methods (Rosenblatt used first two)
 1. Simply initialize convolutional kernels randomly
 2. Design them by hand example, by setting each kernel to detect edges at a certain orientation or scale.
 3. Learn the kernels using unsupervised methods - apply k -means clustering to small image patches, then use each learned centroid as a convolution kernel

10. The Neuroscientific Basis for Convolutional Networks

- Convolutional networks may be the greatest success story of biologically inspired AI
- Some of the key design principles of neural networks were drawn from neuroscience
- Hubel and Wiesel won the Nobel prize in 1981 for their work on the cat's visual system 1960s-1970s - They observed how neurons in the cat's brain responded to images projected in precise locations on a screen in front of the cat. **Their great discovery was that neurons in the early visual system responded most strongly to very specific patterns of light, such as precisely oriented bars, but responded hardly at all to other patterns**

10. The Neuroscientific Basis for Convolutional Networks

- Neurons in the retina perform simple processing, don't change image representation
- Image passes through the optic nerve to a brain region called the lateral geniculate body
- Signal then reaches visual cortex area V1
 - V1 also called the **primary visual cortex**
 - The first area of the brain that performs advanced processing of visual input
 - Located at the back of the head

10. The Neuroscientific Basis for Convolutional Networks

- V1 properties captured in convolutional nets
 - V1 has a 2D structure mirroring the retina image
 - V1 contains many **simple cells**
 - Each characterized by a linear function of the image in a small, spatially localized receptive field
 - V1 contains many **complex cells**
 - These cells respond to features similar to the simple cells
 - But invariant to small shifts in the position of the feature
 - Inspired pooling strategies such as maxout units

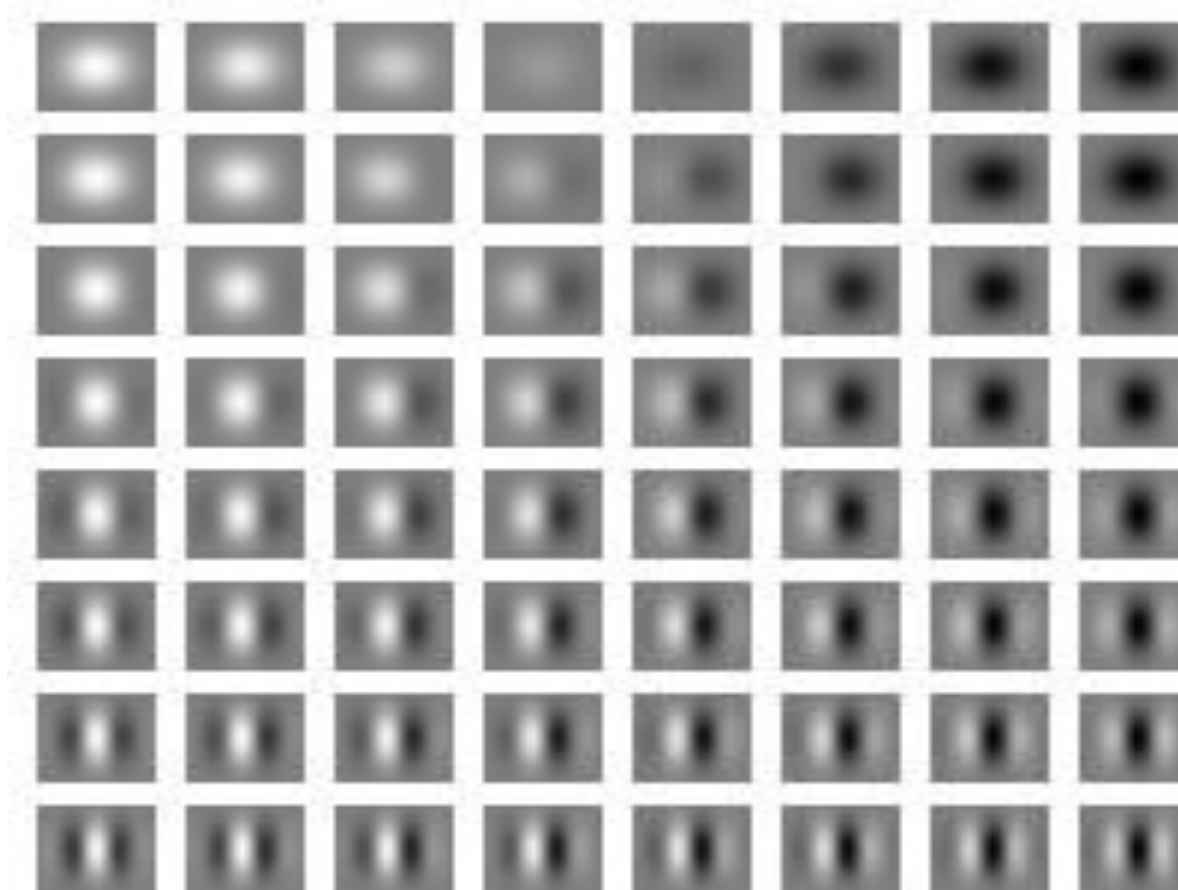
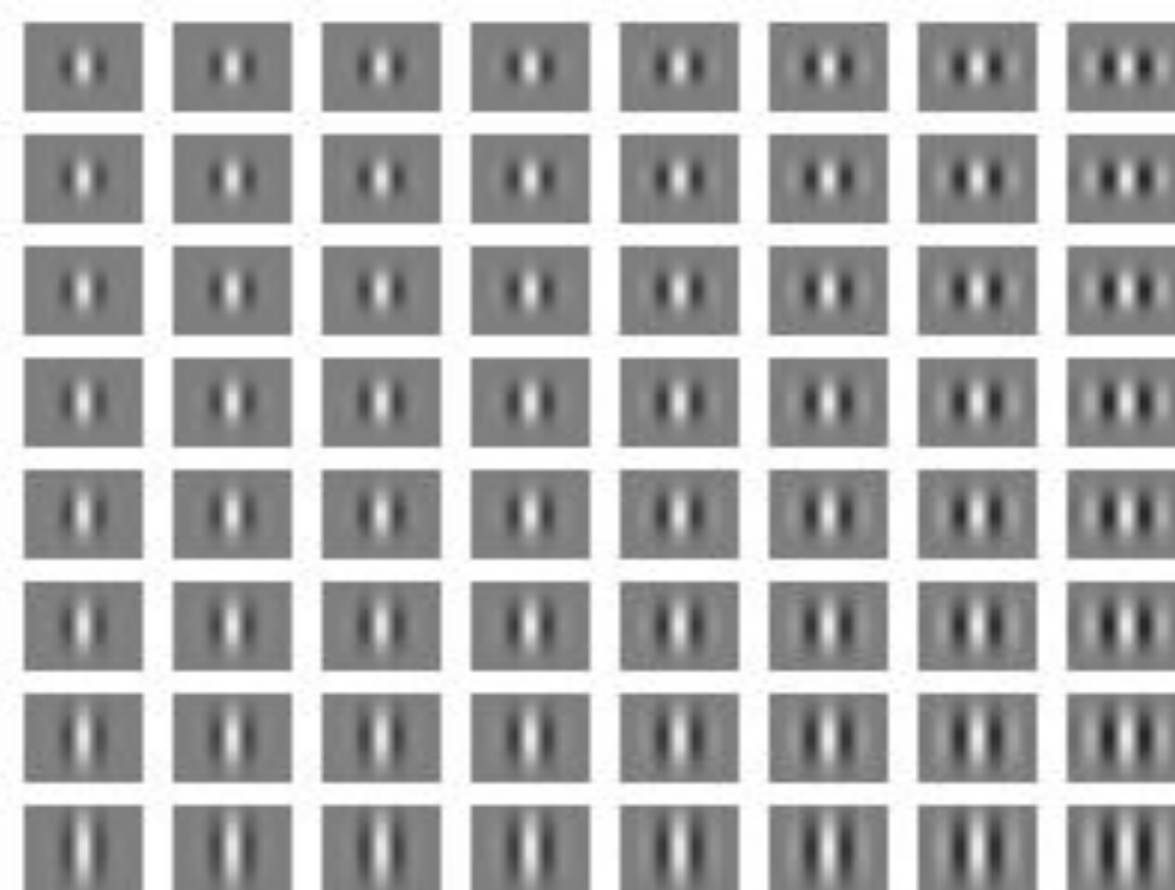
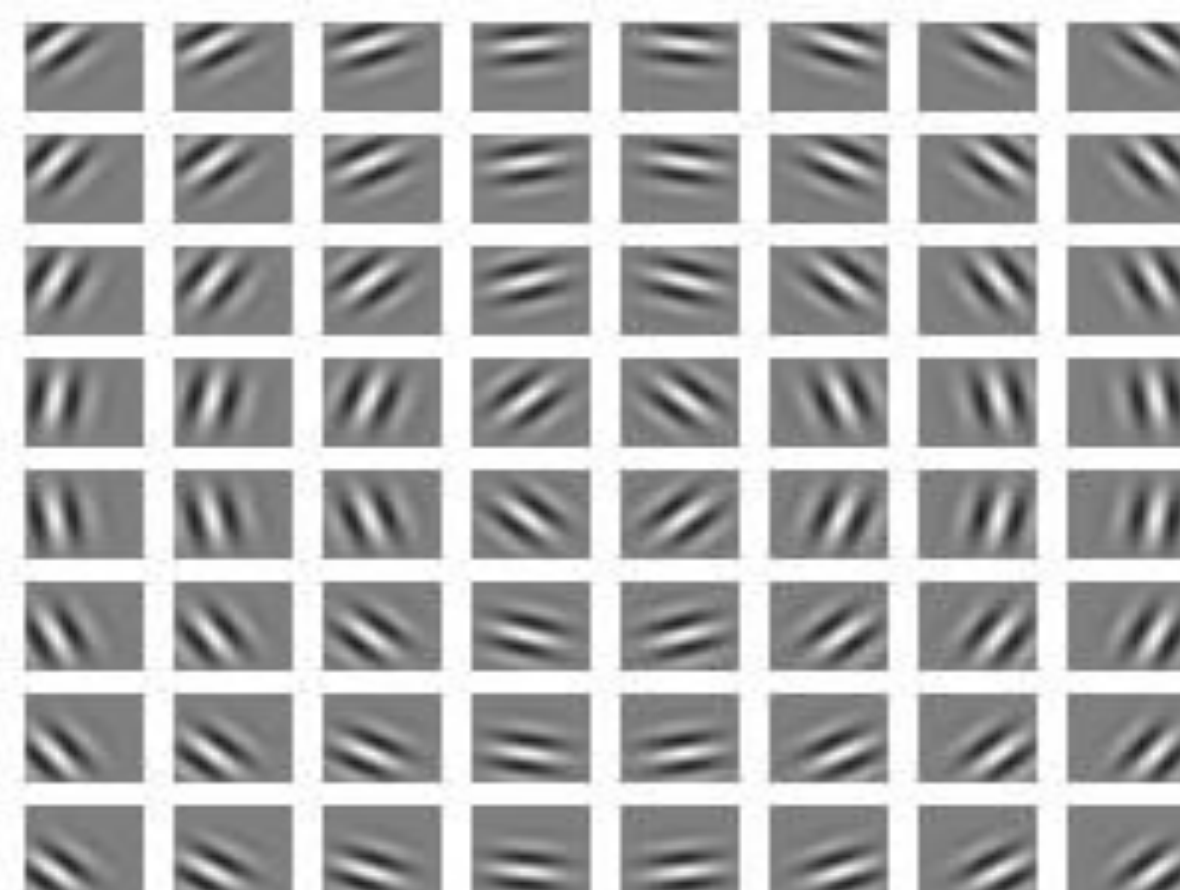
10. The Neuroscientific Basis for Convolutional Networks

- Although we know most about area V1, we believe similar principles apply to other areas
 - Basic strategy of detection followed by pooling
- Passing through deeper layers, we find cells responding to specific concepts
 - These cells are nicknamed “grandmother cells”
 - The idea being that a neuron activates upon seeing their grandmother anywhere in the image

10. The Neuroscientific Basis for Convolutional Networks

- Reverse correlation
 - In biological networks we don't have access to the weights themselves
 - However, we can put an electrode in a neuron, display images in front of the animal's retina, and record the activation of the neuron
 - We can then fit a linear model to these responses to approximate the neuron's weights
- Most V1 cells have weights of Gabor functions

Gabor Functions



White = positive weight, black = negative, gray = zero weight

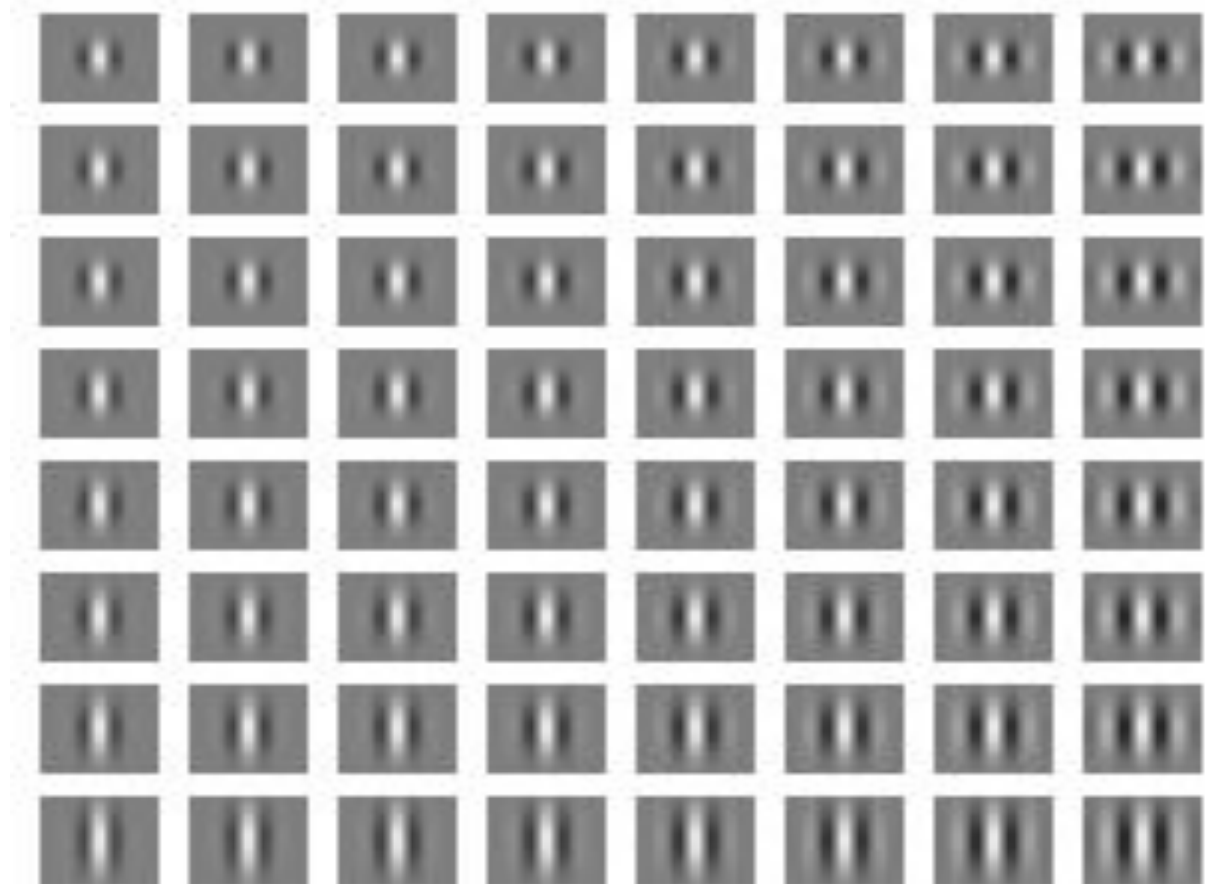
(Left) detectors in different orientations, (Center) detectors of increasing width and height, (Right) different sinusoidal params

Gabor functions with a variety of parameter settings. White indicates large positive weight, black indicates large negative weight, and the background gray corresponds to zero weight.

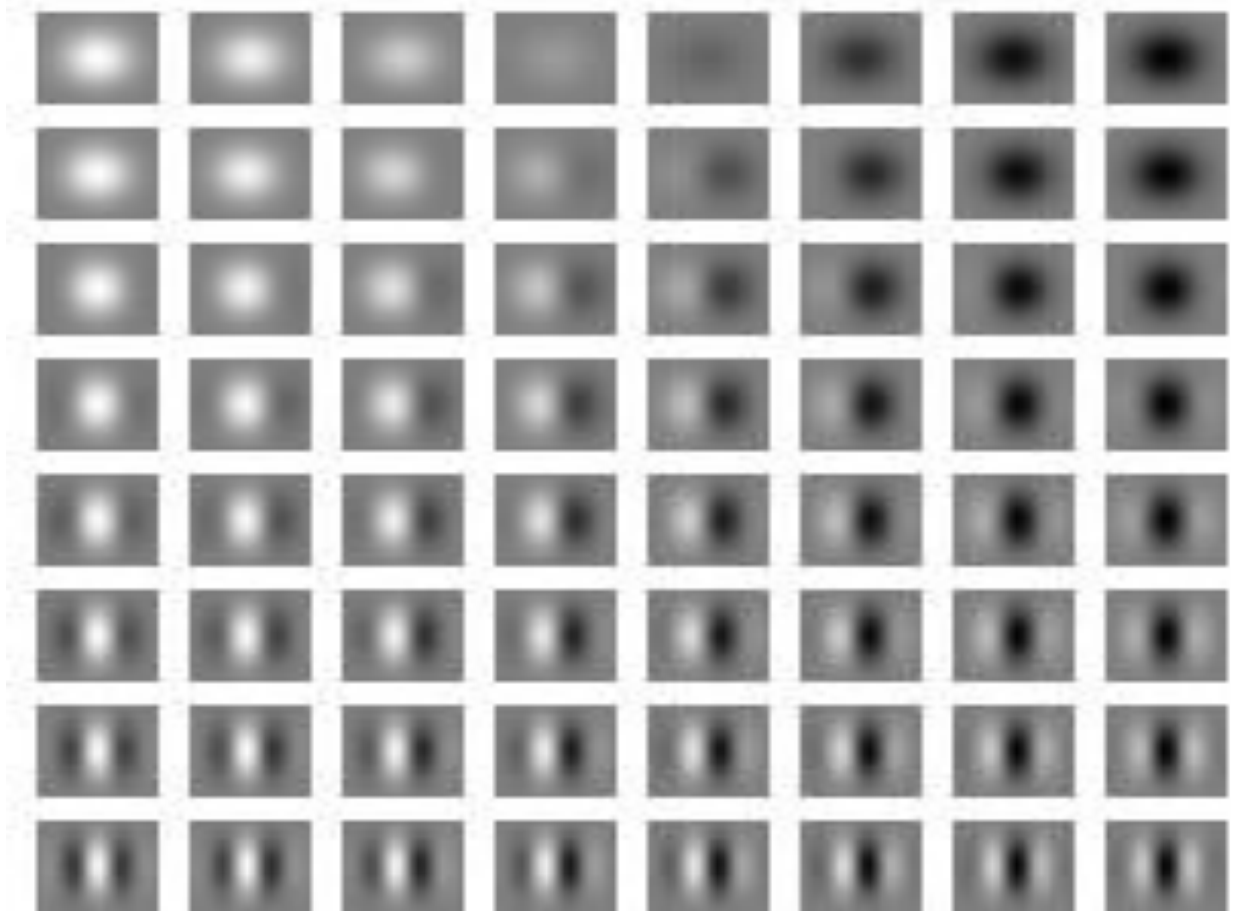
corresponds to zero weight. *(Left)* Gabor functions with different values of the parameters that control the coordinate system: x_0 , y_0 , and τ . Each Gabor function in this grid is assigned a value of x_0 and y_0 proportional to its position in its grid, and τ is chosen so that each Gabor filter is sensitive to the direction radiating out from the center of the grid. For the other two plots, x_0 , y_0 , and τ are fixed to zero. *(Center)* Gabor functions with different Gaussian scale parameters β_x and β_y . Gabor functions are arranged in increasing width (decreasing β_x) as we move left to right through the grid, and increasing height (decreasing β_y) as we move top to bottom. For the other two plots, the β values are fixed to 1.5 times the image width. *(Right)* Gabor functions with different sinusoid parameters f and ϕ . As we move top to bottom, f increases, and as we move left to right, ϕ increases. For the other two plots, ϕ is fixed to 0 and f is fixed to 5 times the image width.

Gabor functions with a variety of parameter settings. White indicates large positive weight, black indicates large negative weight, and the background gray corresponds to zero weight.

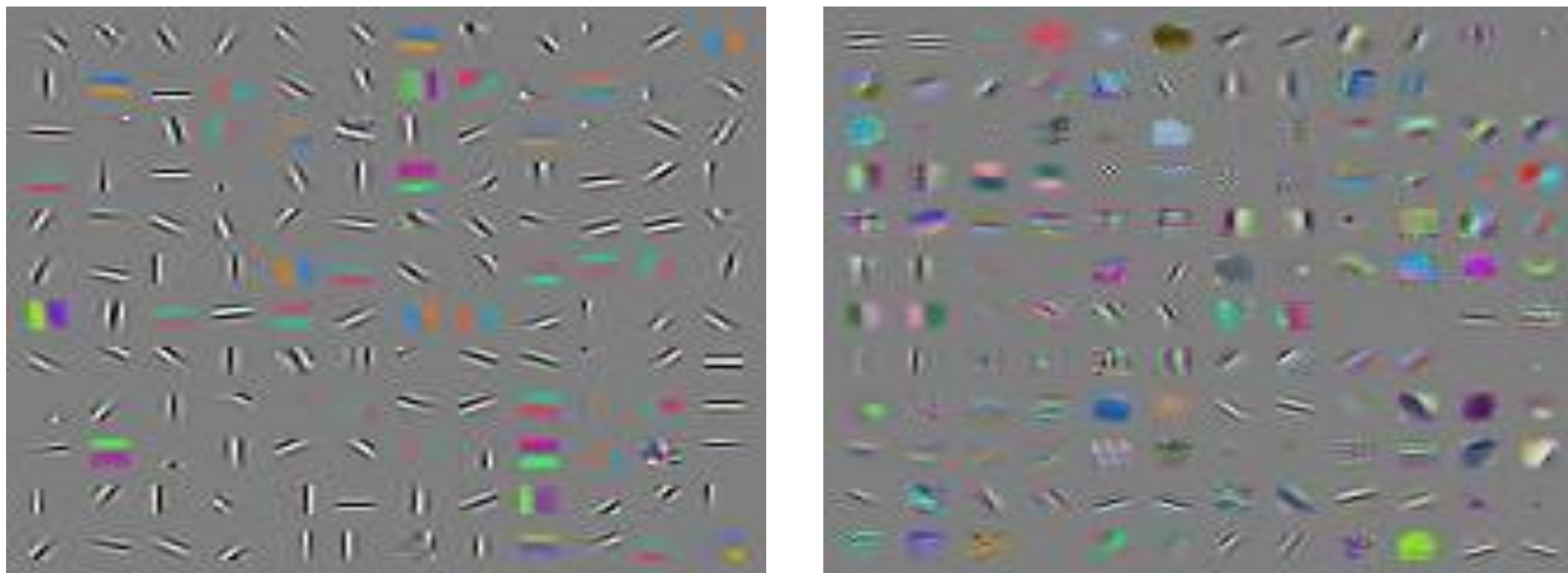
corresponds to zero weight. *(Left)* Gabor functions with different values of the parameters that control the coordinate system: x_0 , y_0 , and τ . Each Gabor function in this grid is assigned a value of x_0 and y_0 proportional to its position in its grid, and τ is chosen so that each Gabor filter is sensitive to the direction radiating out from the center of the grid. For the other two plots, x_0 , y_0 , and τ are fixed to zero. *(Center)* Gabor functions with different Gaussian scale parameters β_x and β_y . Gabor functions are arranged in increasing width (decreasing β_x) as we move left to right through the grid, and increasing height (decreasing β_y) as we move top to bottom. For the other two plots, the β values are fixed to 1.5 times the image width. *(Right)* Gabor functions with different sinusoid parameters f and ϕ . As we move top to bottom, f increases, and as we move left to right, ϕ increases. For the other two plots, ϕ is fixed to 0 and f is fixed to 5 times the image width.



corresponds to zero weight. *(Left)* Gabor functions with different values of the parameters that control the coordinate system: x_0 , y_0 , and τ . Each Gabor function in this grid is assigned a value of x_0 and y_0 proportional to its position in its grid, and τ is chosen so that each Gabor filter is sensitive to the direction radiating out from the center of the grid. For the other two plots, x_0 , y_0 , and τ are fixed to zero. *(Center)* Gabor functions with different Gaussian scale parameters β_x and β_y . Gabor functions are arranged in increasing width (decreasing β_x) as we move left to right through the grid, and increasing height (decreasing β_y) as we move top to bottom. For the other two plots, the β values are fixed to 1.5 times the image width. *(Right)* Gabor functions with different sinusoid parameters f and ϕ . As we move top to bottom, f increases, and as we move left to right, ϕ increases. For the other two plots, ϕ is fixed to 0 and f is fixed to 5 times the image width.



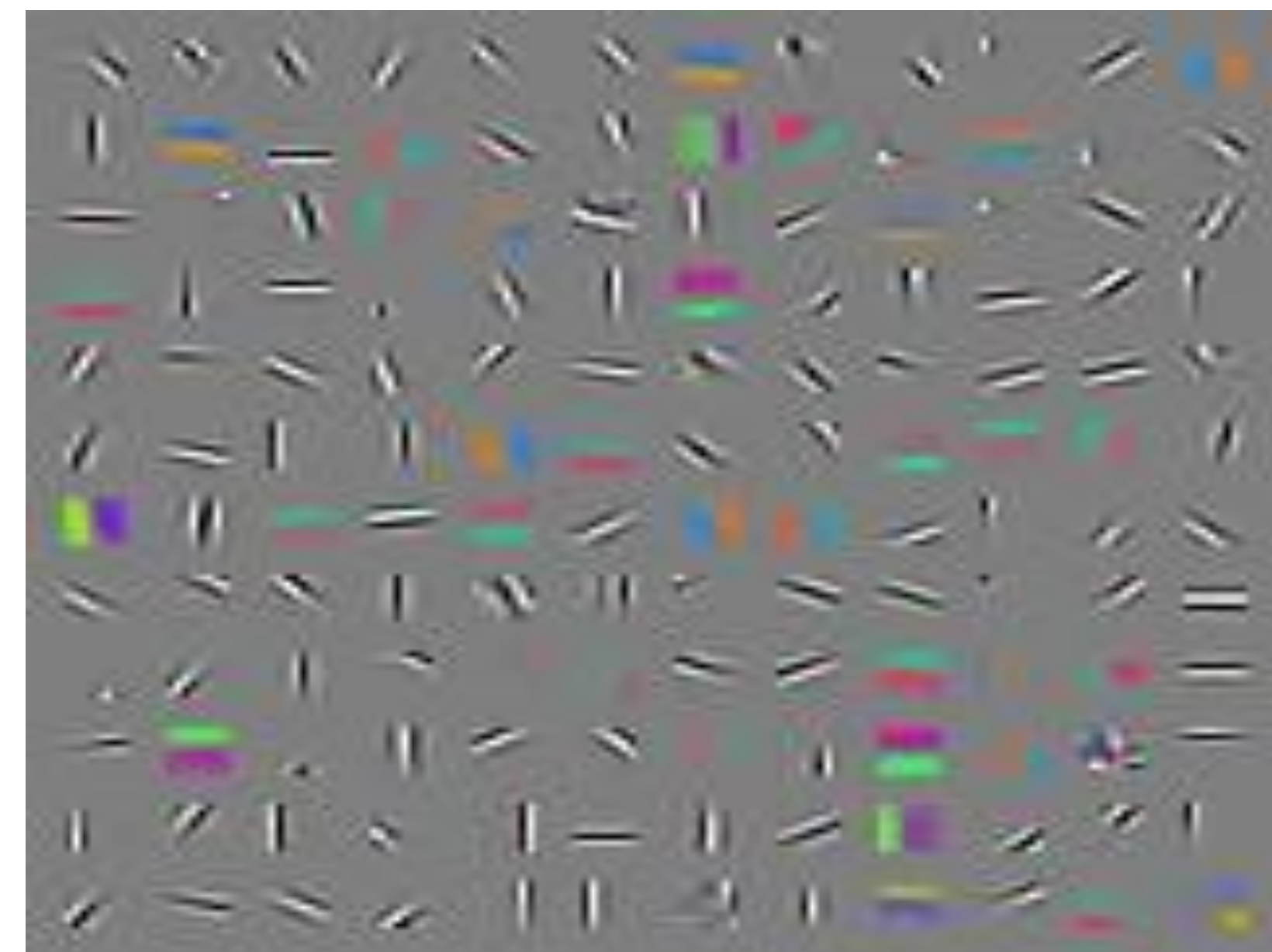
Gabor-like Learned Kernels



(Left) Weights learned by unsupervised learning
(Right) Convolutional kernels learned by first layer of fully supervised convolutional maxout network

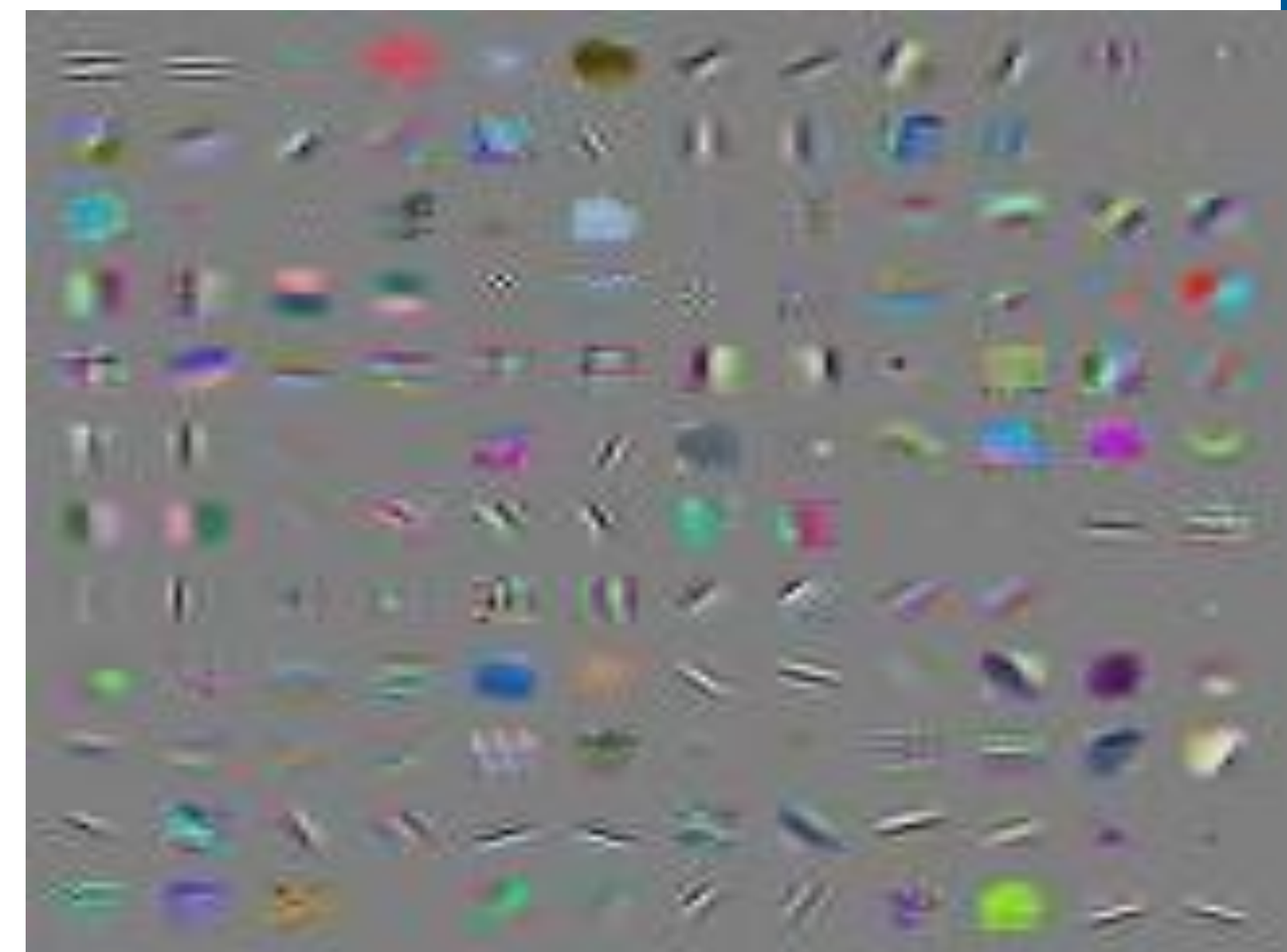
Gabor-like Learned Kernels

Figure 9.19: Many machine learning algorithms learn features that detect edges or specific colors of edges when applied to natural images. These feature detectors are reminiscent of the Gabor functions known to be present in the primary visual cortex. *(Left)*Weights learned by an unsupervised learning algorithm (spike and slab sparse coding) applied to small image patches. *(Right)*Convolution kernels learned by the first layer of a fully supervised convolutional maxout network. Neighboring pairs of filters drive the same maxout unit.



Gabor-like Learned Kernels

Figure 9.19: Many machine learning algorithms learn features that detect edges or specific colors of edges when applied to natural images. These feature detectors are reminiscent of the Gabor functions known to be present in the primary visual cortex. *(Left)*Weights learned by an unsupervised learning algorithm (spike and slab sparse coding) applied to small image patches. *(Right)*Convolution kernels learned by the first layer of a fully supervised convolutional maxout network. Neighboring pairs of filters drive the same maxout unit.



11. Convolutional Networks and the History of Deep Learning

- Convolutional networks have played an important role in the history of deep learning
 - Application of neuroscience to machine learning
 - First deep models to perform well
 - First important commercial applications For example, in the 1990s, the neural network research group at AT&T developed a convolutional network for reading checks (LeCun et al., 1998b)
 - Used to win many contests - Krizhevsky et al. (2012) won the ImageNet object recognition challenge
 - Some of first deep networks trained with back-prop Performed well decades ago to pave the way toward acceptance of neural networks in general.
 - Convolutional networks provide a way to specialize neural networks to work with data that has a clear grid-structured topology and to scale such models to very large size.

11. Convolutional Networks and the History of Deep Learning

- Convolutional networks allow specialized neural networks for grid-structured topology
 - Most successful on 2D image topology
 - For 1D sequential data we use recurrent networks



**RV College of
Engineering®**

Go, change the world



**RV College of
Engineering®**

Go, change the world



**RV College of
Engineering®**

Go, change the world

Thank You

